

# CSE 4303

# Data Structures

Topic: Heap Basics

Sabbir Ahmed

Assistant Prof | CSE | IUT

sabbirahmed@iut-dhaka.edu

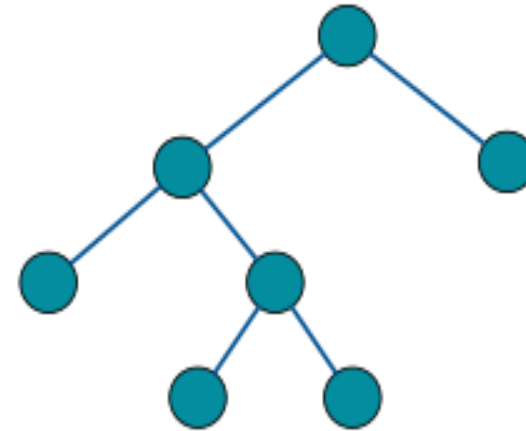
*Heap can be viewed as a  
complete binary tree, which satisfies.....*

*Heap can be viewed as a complete binary tree, which satisfies.....*

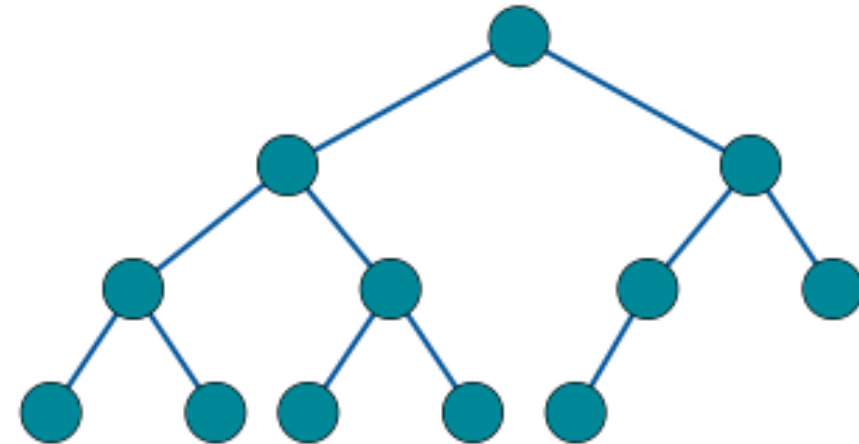
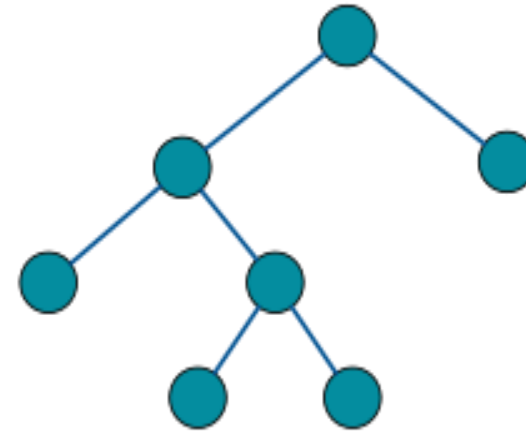
*Heap can be viewed as a  
complete **binary tree**, which satisfies.....*

- **Binary tree:** each node has at most two children (left child & right child).

- **Binary tree:** each node has at most two children (left child & right child).



- **Binary tree:** each node has at most two children (left child & right child).



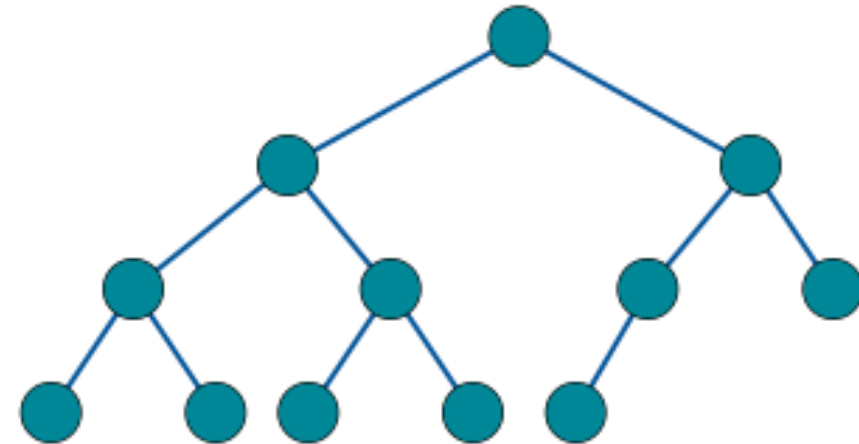
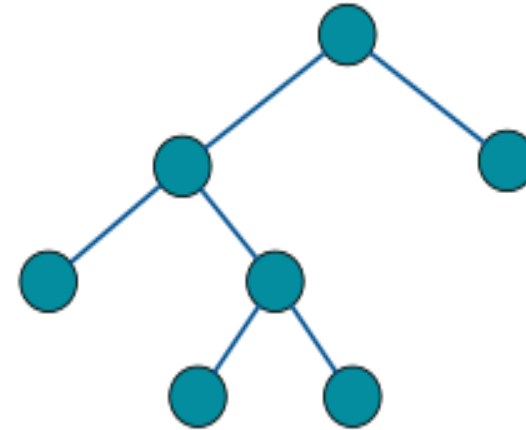
- **Binary tree:** each node has at most two children (left child & right child).
- **Complete binary tree:** filled on all levels except possibly the lowest, which is filled from the left up to a point.
  - Level by Level
  - Left to right



*Which one represents a complete binary tree?*

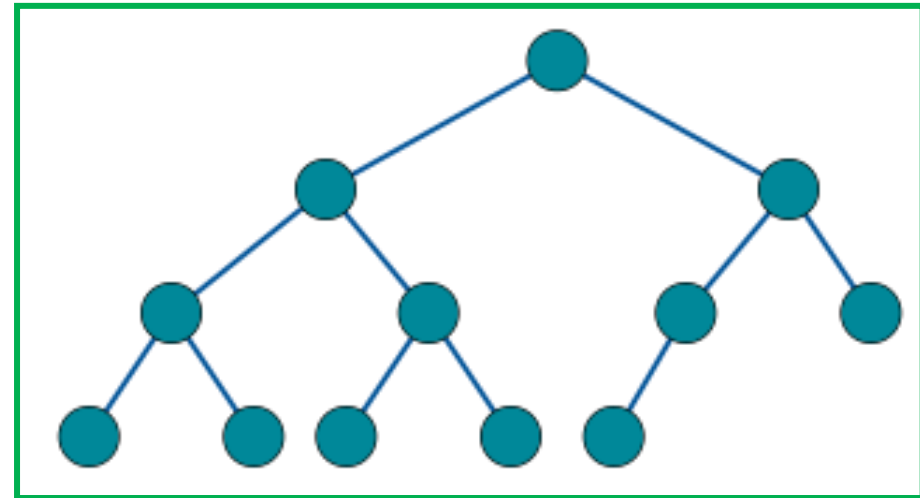
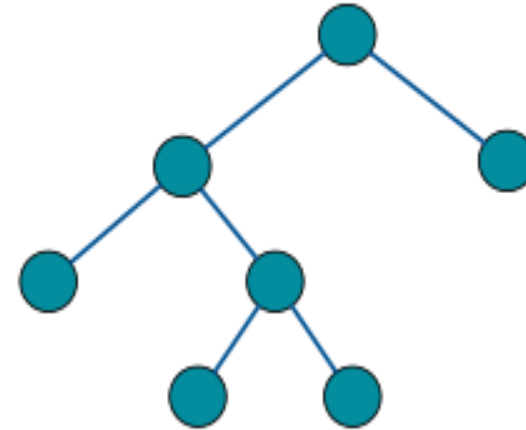
- **Binary tree:** each node has at most two children (left child & right child).
- **Complete binary tree:** filled on all levels except possibly the lowest, which is filled from the left up to a point.

- Level by Level
- Left to right

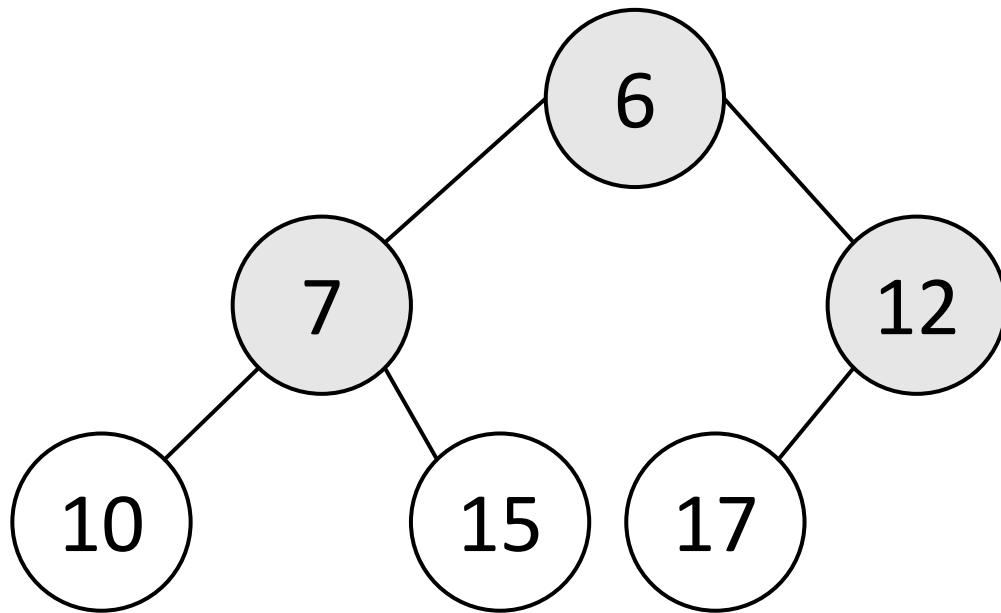


*Which one represents a complete binary tree?*

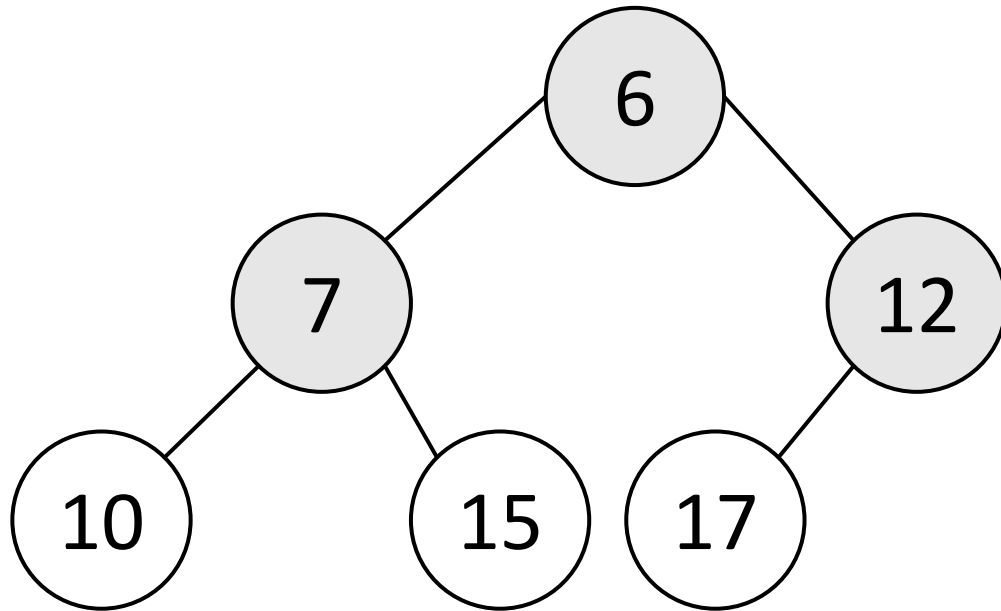
- **Binary tree:** each node has at most two children (left child & right child).
- **Complete binary tree:** filled on all levels except possibly the lowest, which is filled from the left up to a point.
  - Level by Level
  - Left to right



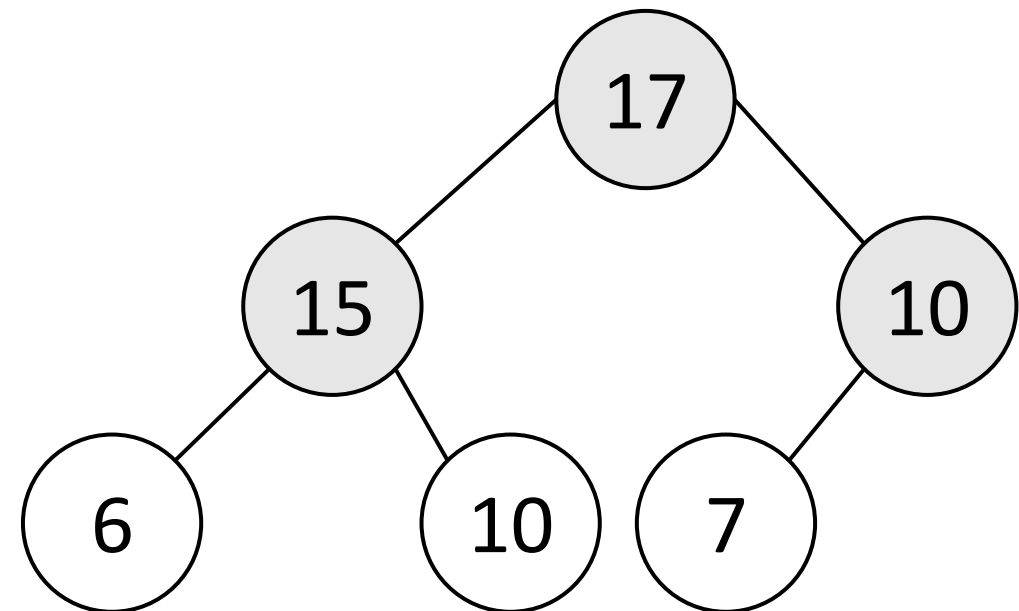
*Heap can be viewed as a  
complete binary tree, which satisfies.....*



*Each parent  $\leq$  children*

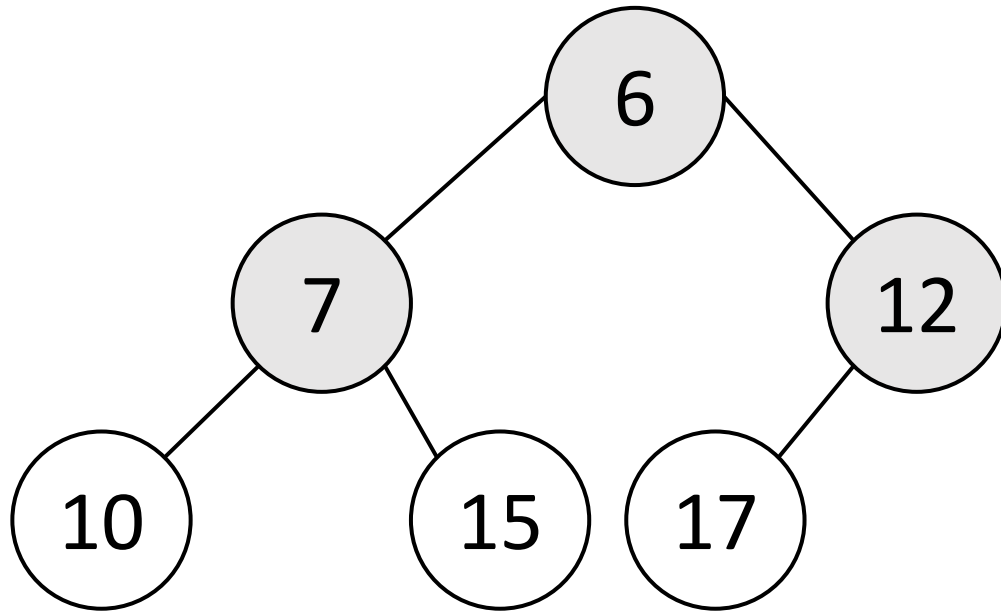


*Each parent  $\leq$  children*



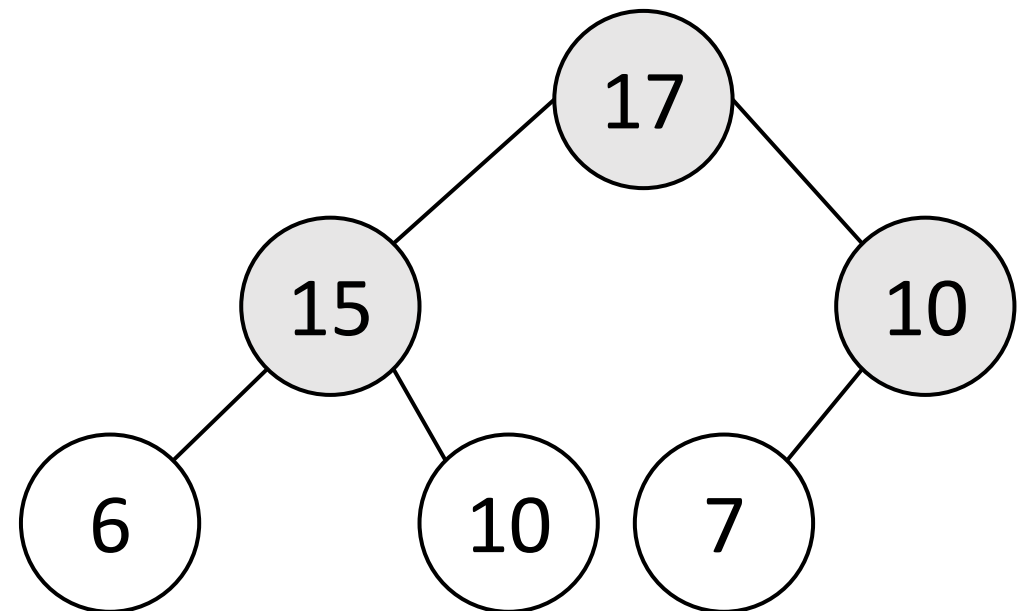
*Each parent  $\geq$  children*

## *min-heap*



*Each parent  $\leq$  children*

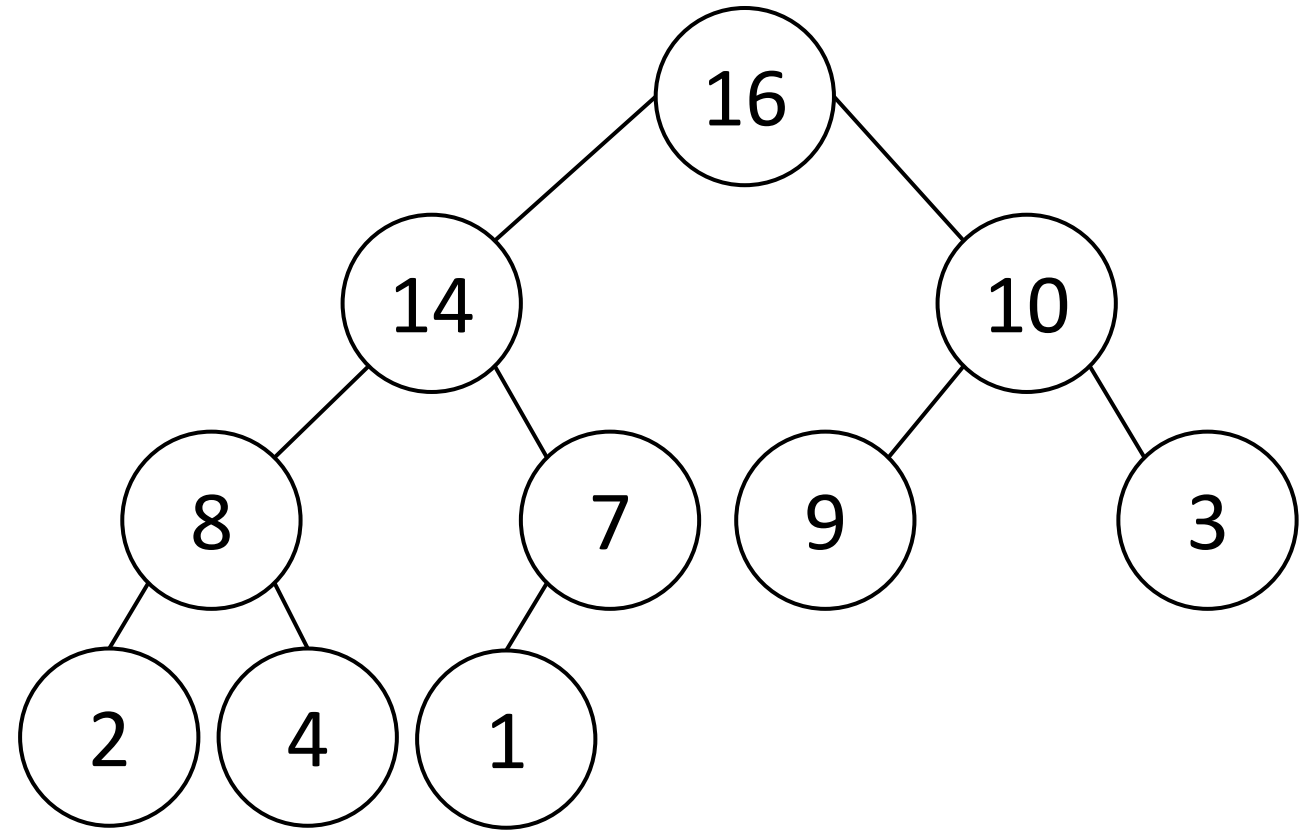
## *max-heap*



*Each parent  $\geq$  children*

*Before diving into heap, lets learn some  
tree terminologies*

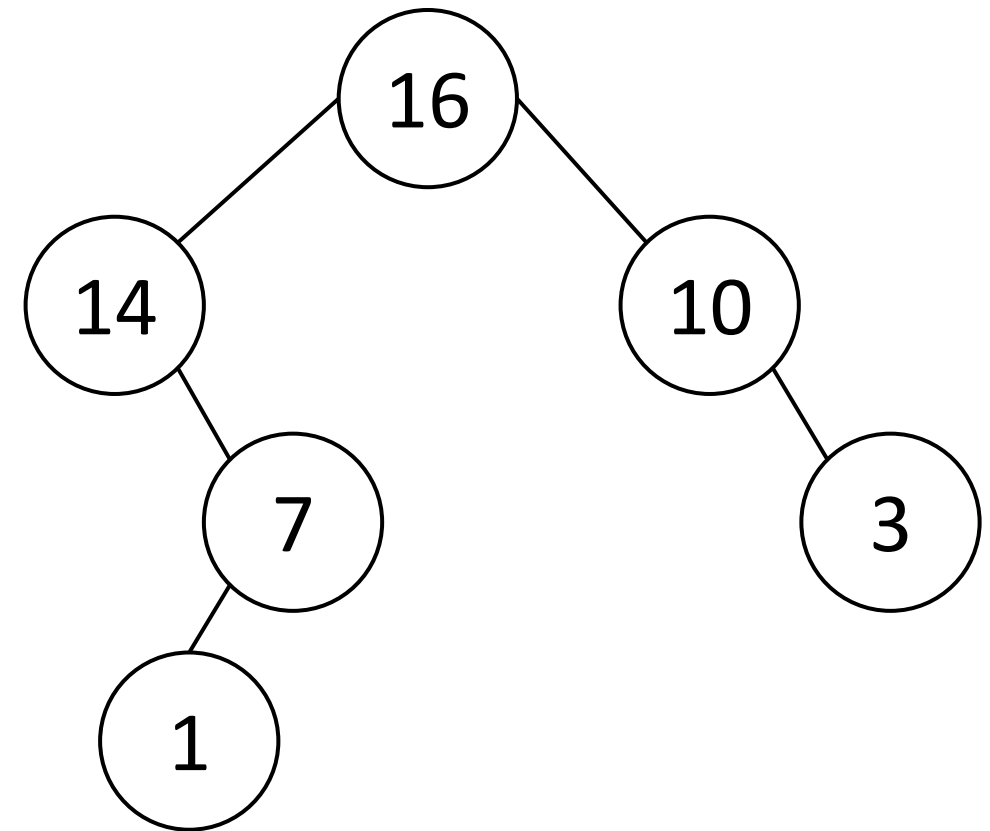
- Binary tree can be stored using array
- Each node of the tree corresponds to an element of the array.



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |



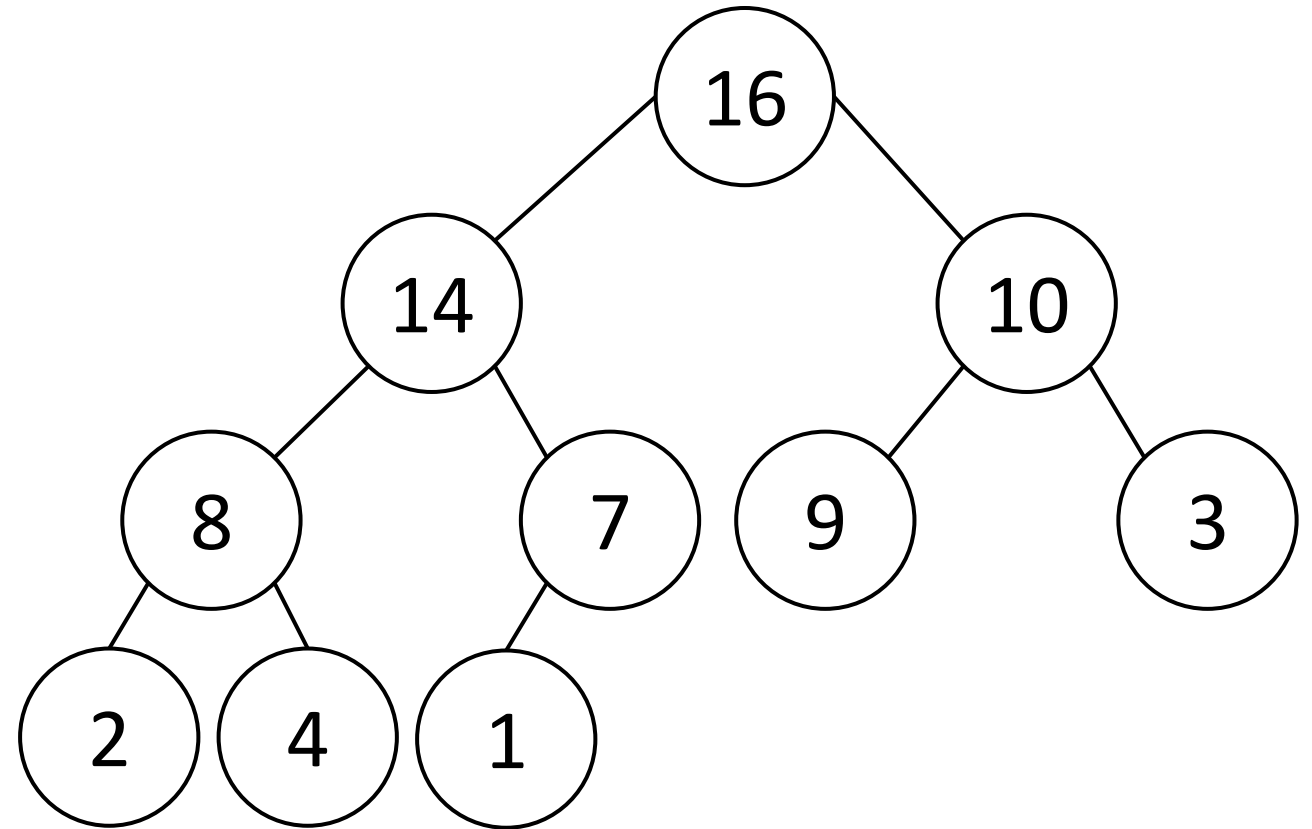
We can store any binary tree using arrays



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 |   | 7 |   | 3 |   |   | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

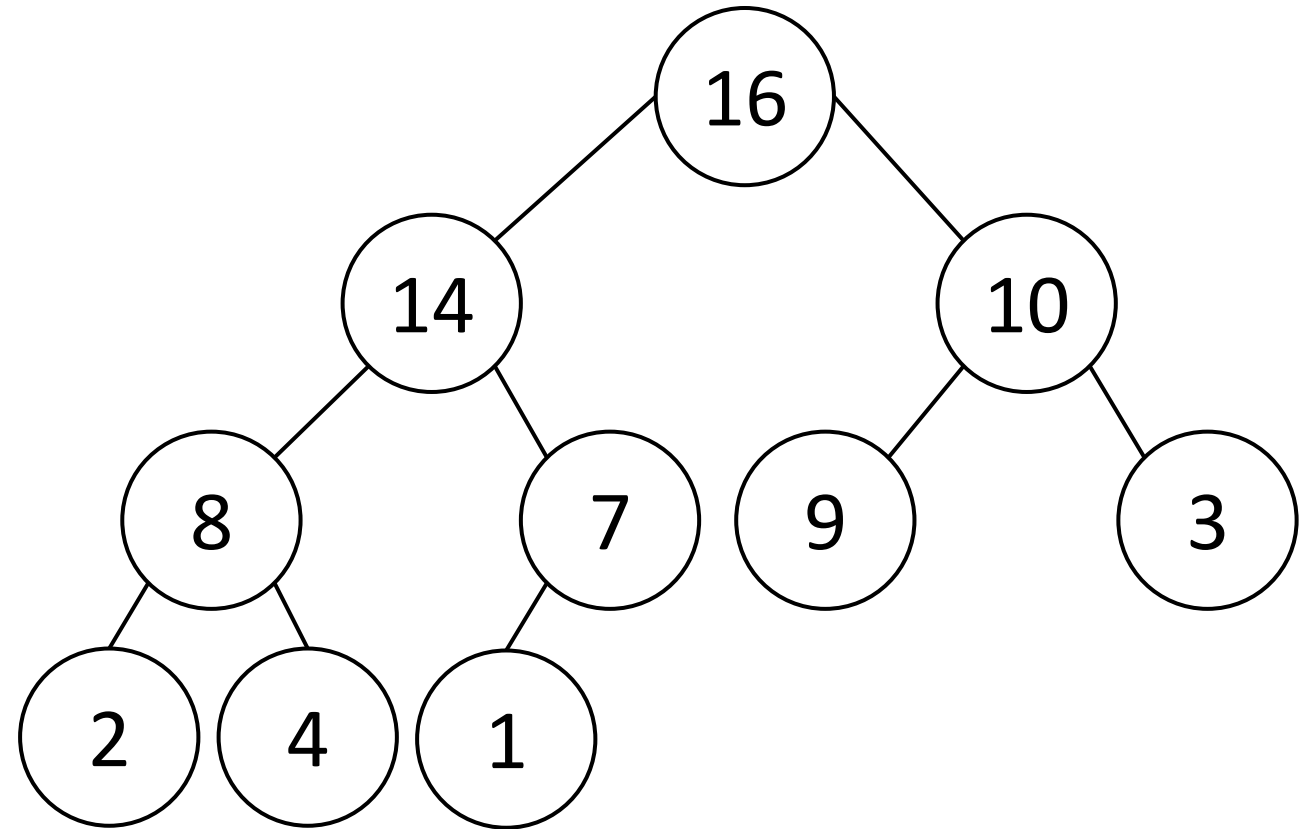
• *leftChild* = ??



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

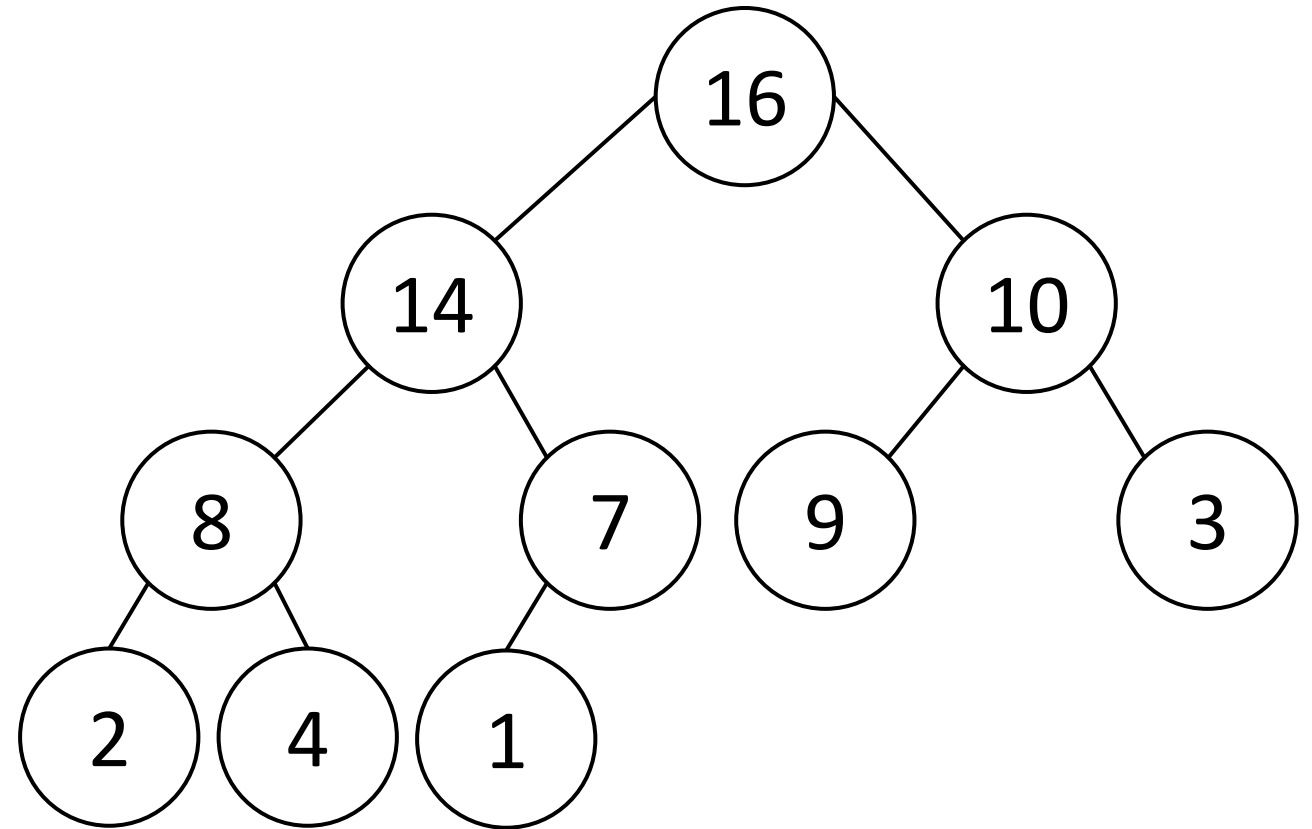
- $leftChild = 2i$



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

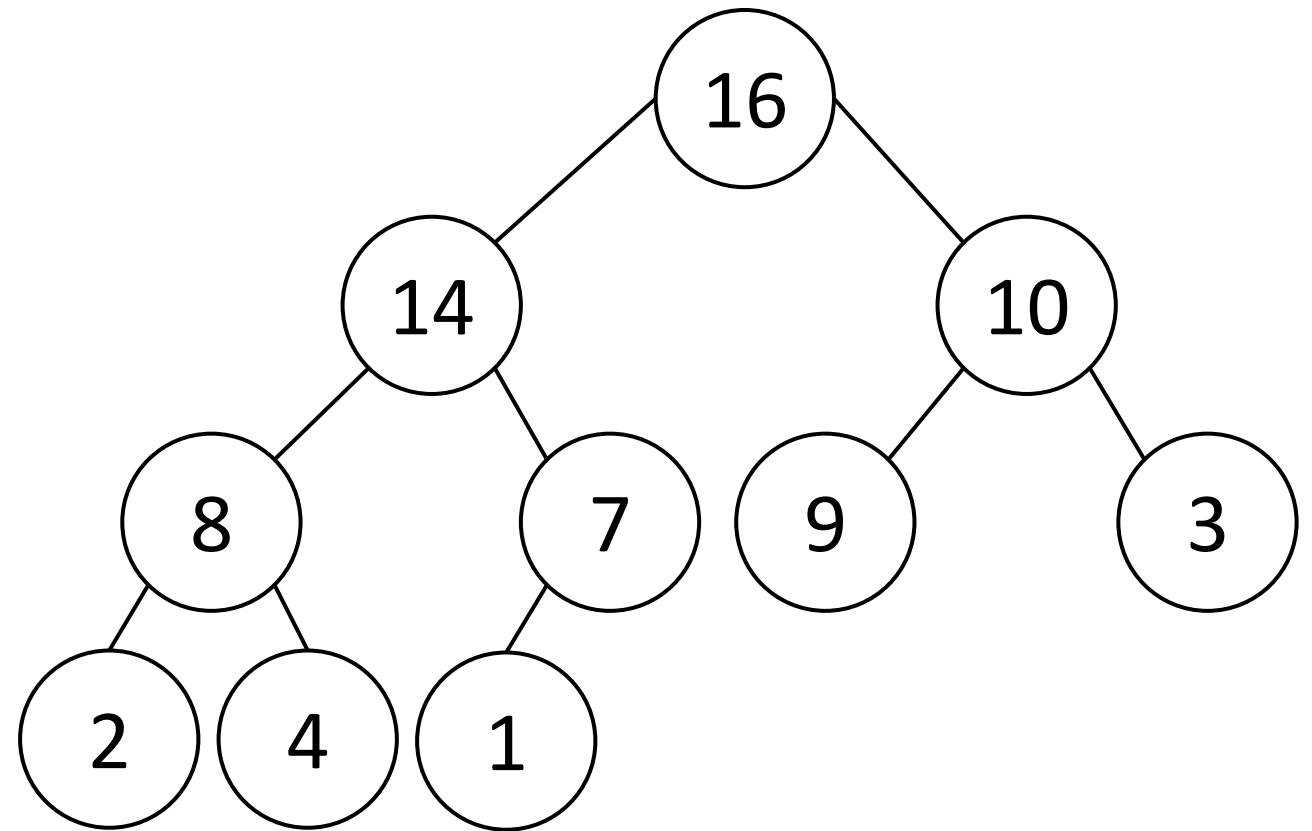
- $leftChild = 2i$
- $rightChild =$  ??



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

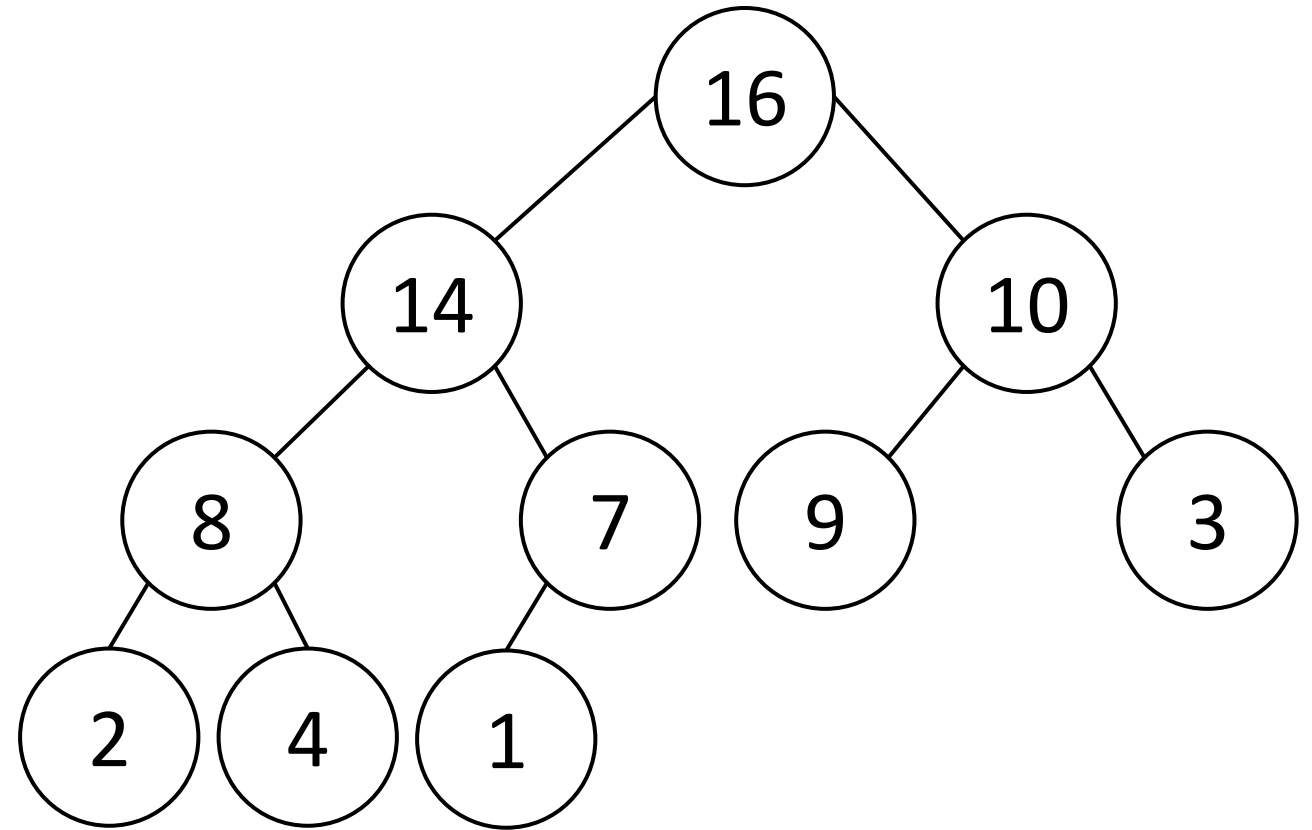
- $leftChild = 2i$
- $rightChild = 2i + 1$



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

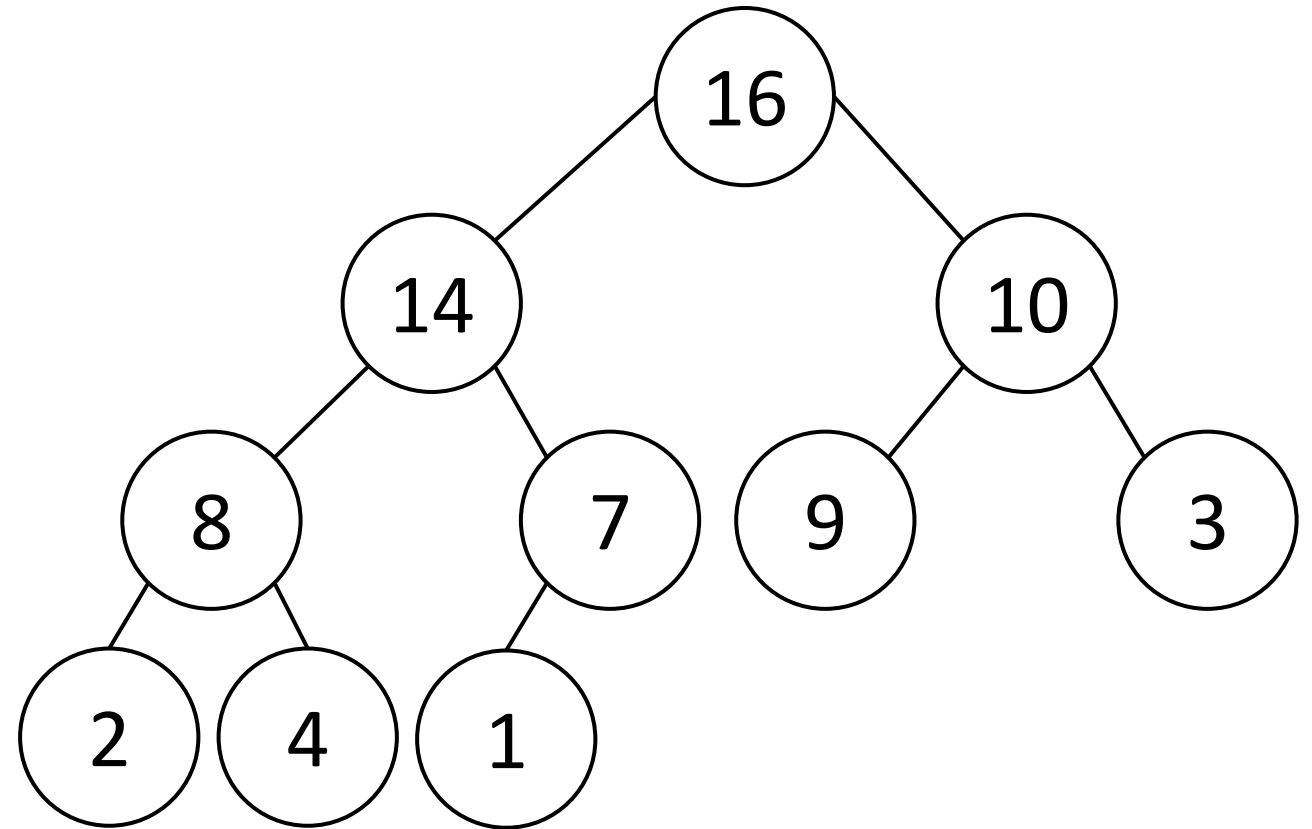
- $leftChild = 2i$
- $rightChild = 2i + 1$
- $parent = ??$



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

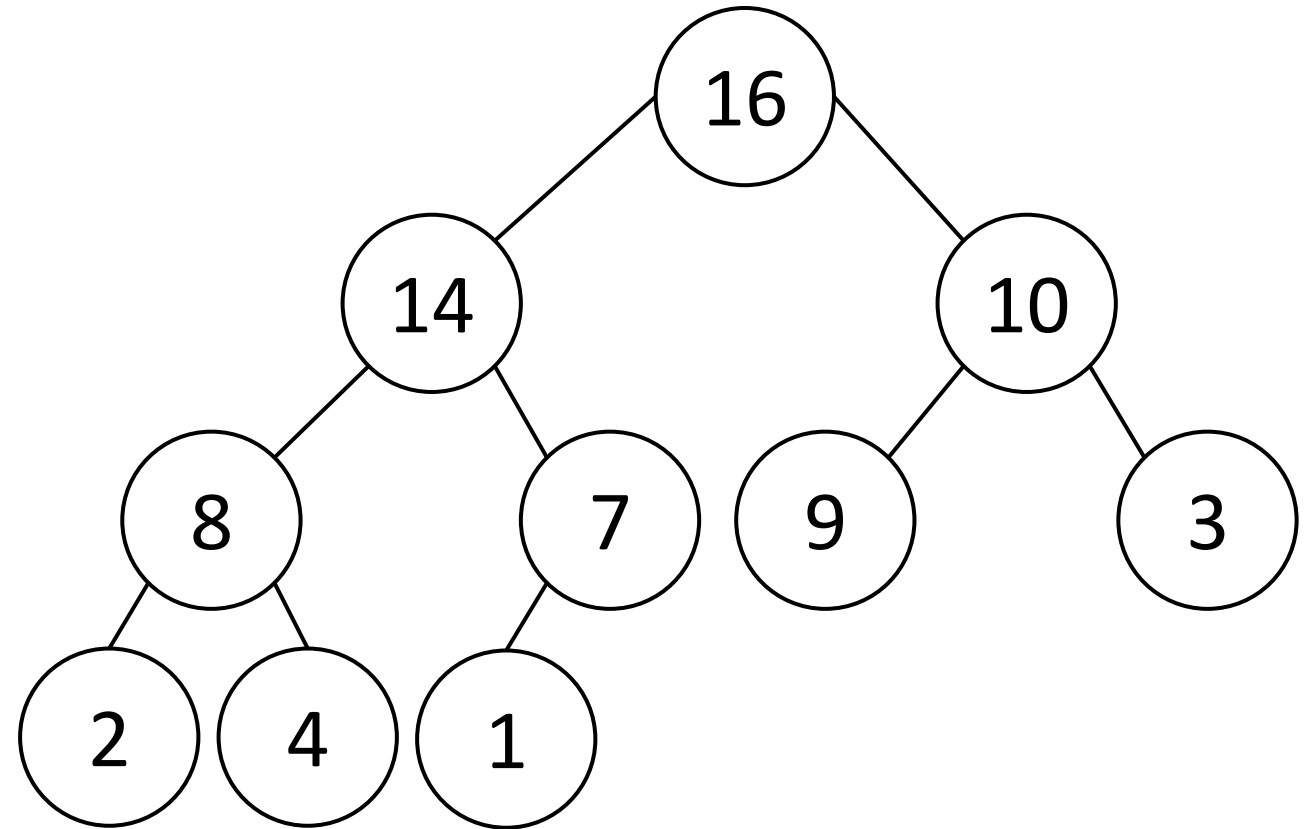
- $leftChild = 2i$
- $rightChild = 2i + 1$
- $parent = floor(\frac{i}{2})$



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

For  $i^{th}$  node:

- $leftChild = 2i$
- $rightChild = 2i + 1$
- $parent = floor(\frac{i}{2})$



*Any pattern in leaf-node index??*

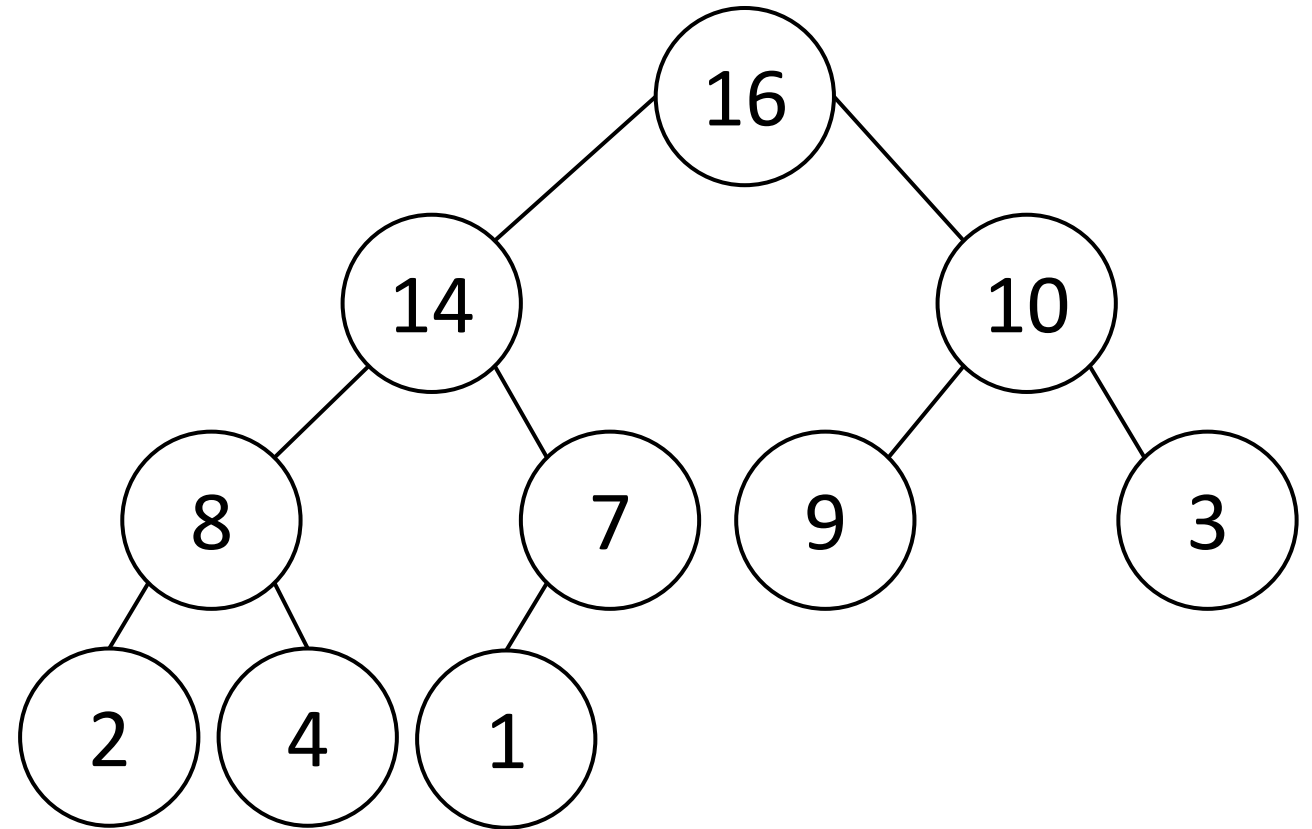
|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |



For  $i^{th}$  node:

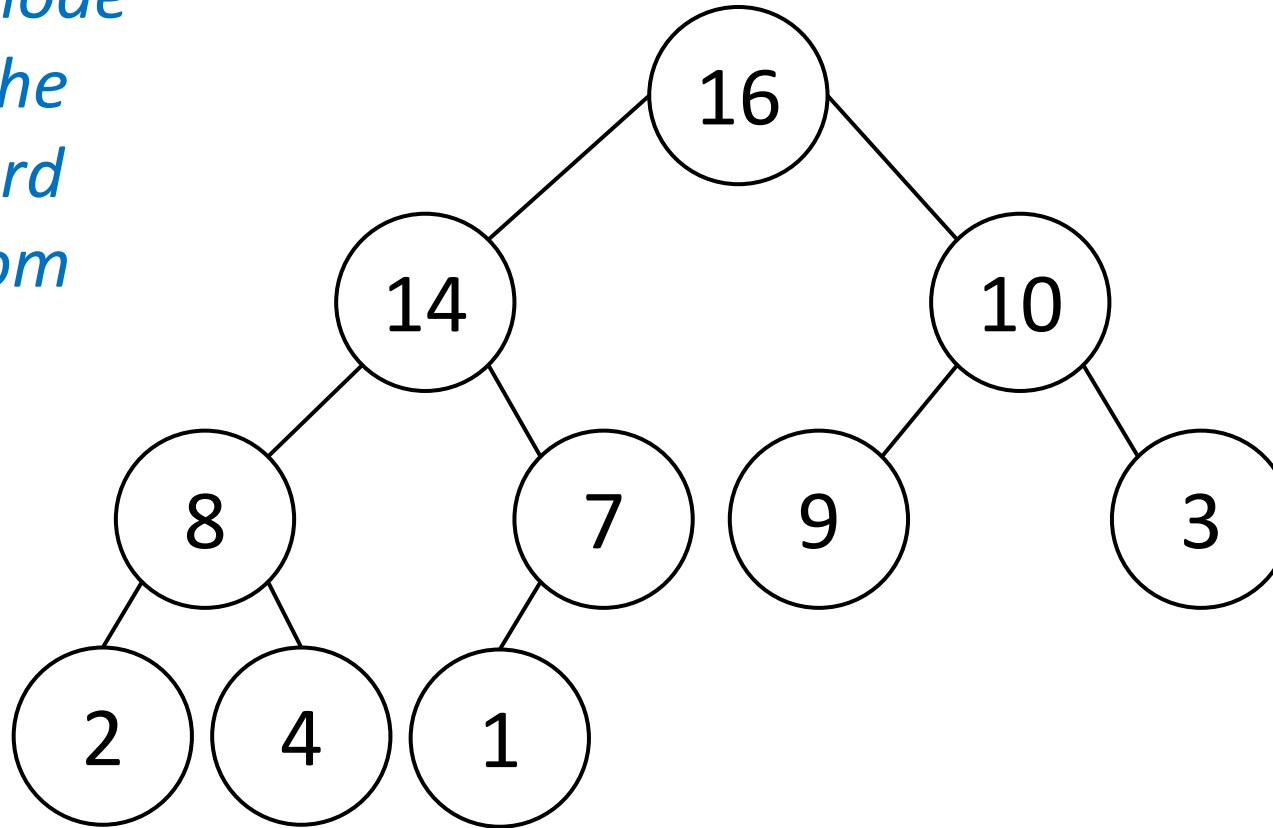
- $leftChild = 2i$
- $rightChild = 2i + 1$
- $parent = floor(\frac{i}{2})$

After  $\frac{n}{2}$  th location, all nodes are leaf node



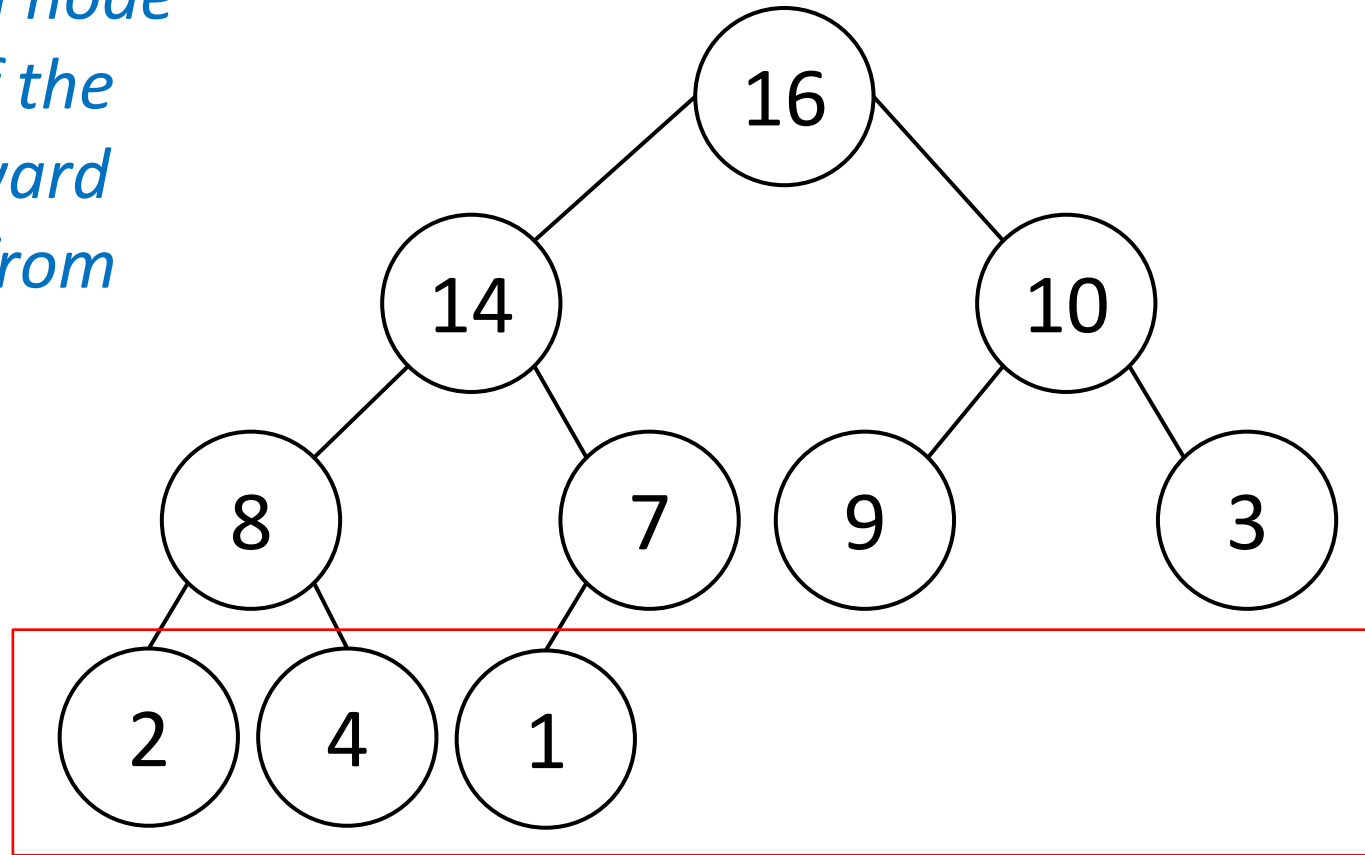
|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **height** of a node is the length of the longest downward path to a leaf from that node



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

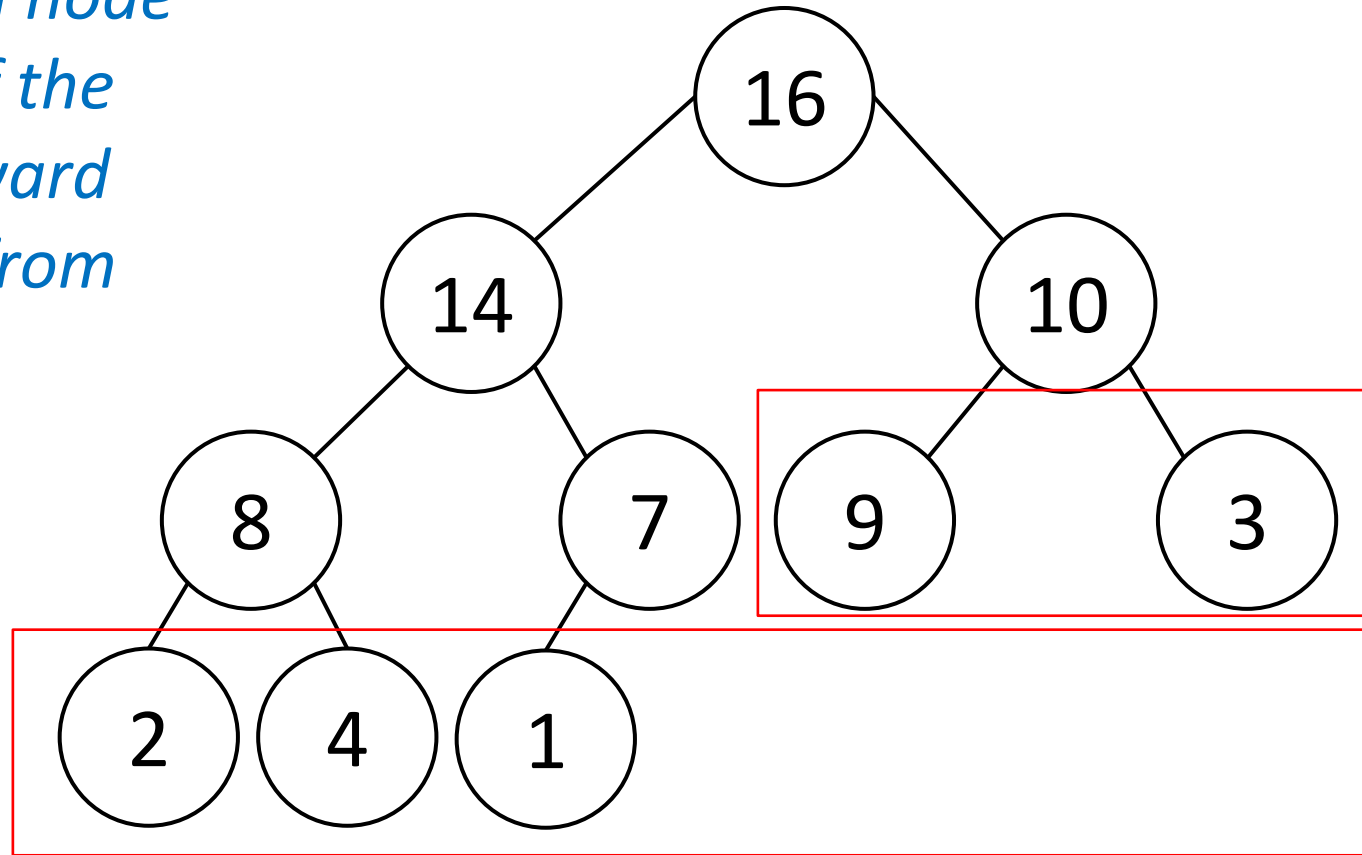
The **height** of a node is the length of the longest downward path to a leaf from that node



*height=0*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **height** of a node is the length of the longest downward path to a leaf from that node

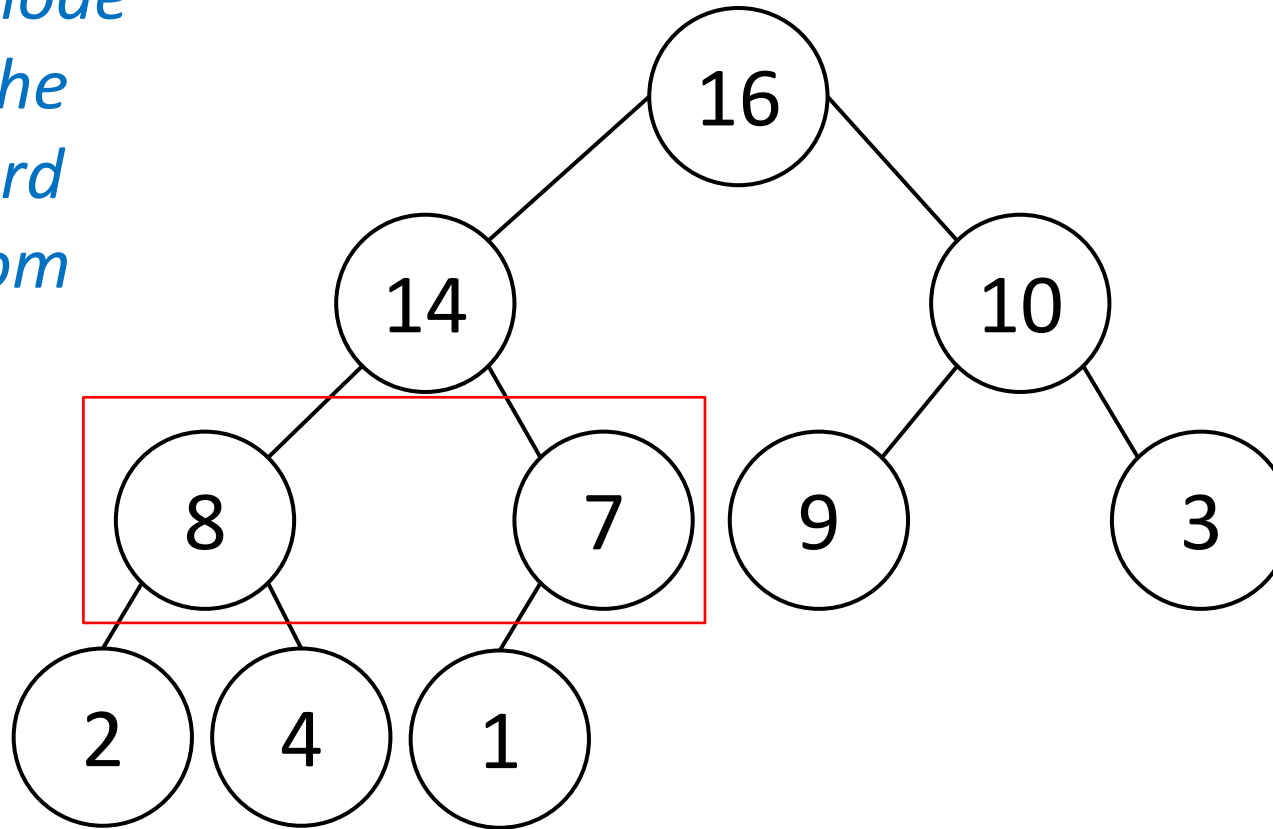


*height=0*

*height=0*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

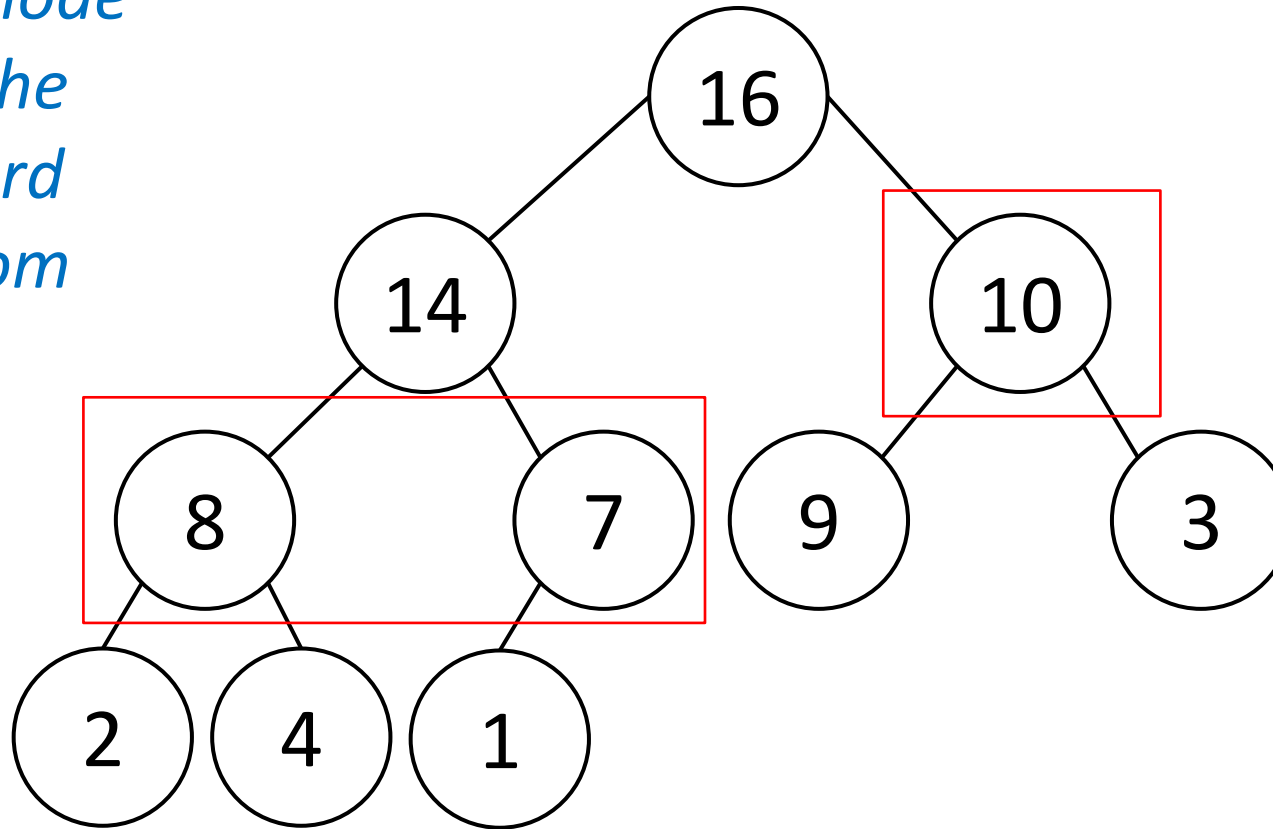
The **height** of a node is the length of the longest downward path to a leaf from that node



*height=1*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

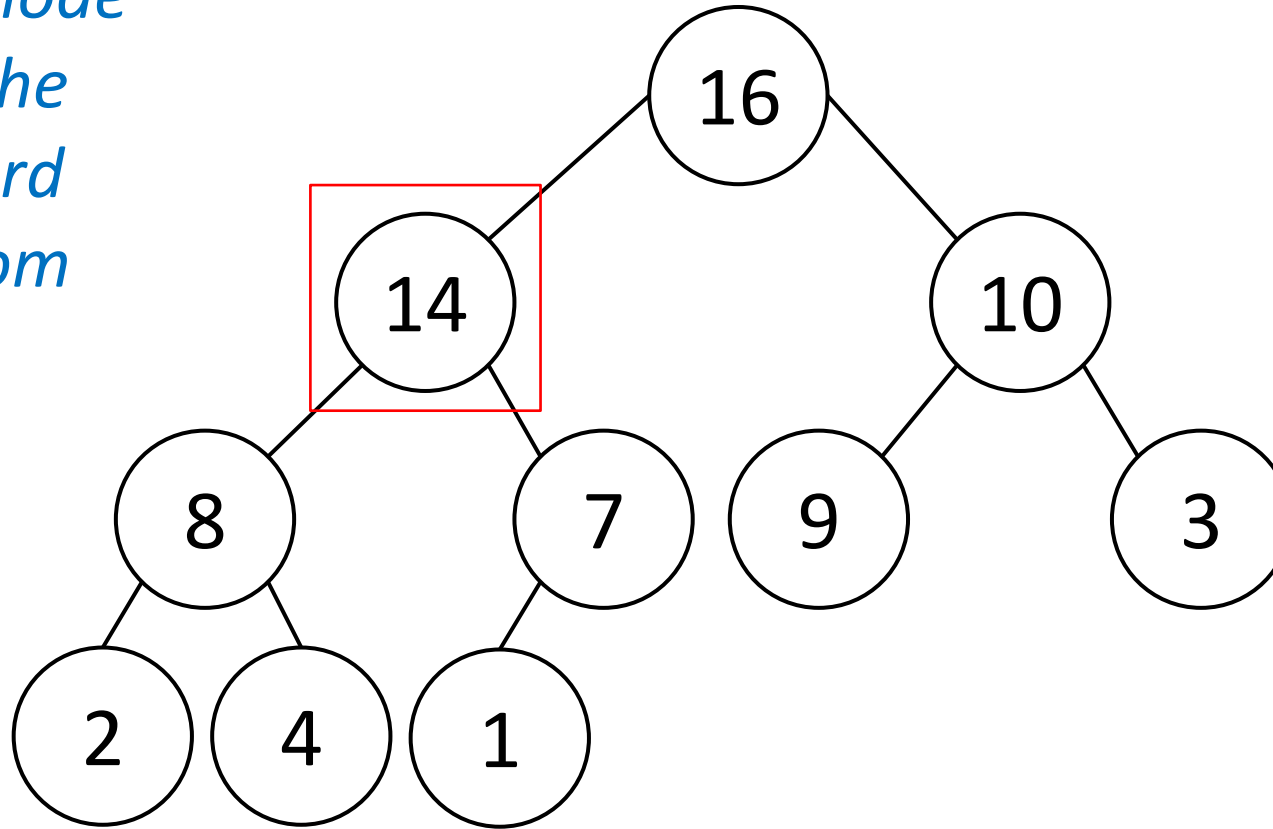
The **height** of a node is the length of the longest downward path to a leaf from that node



*height=1*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

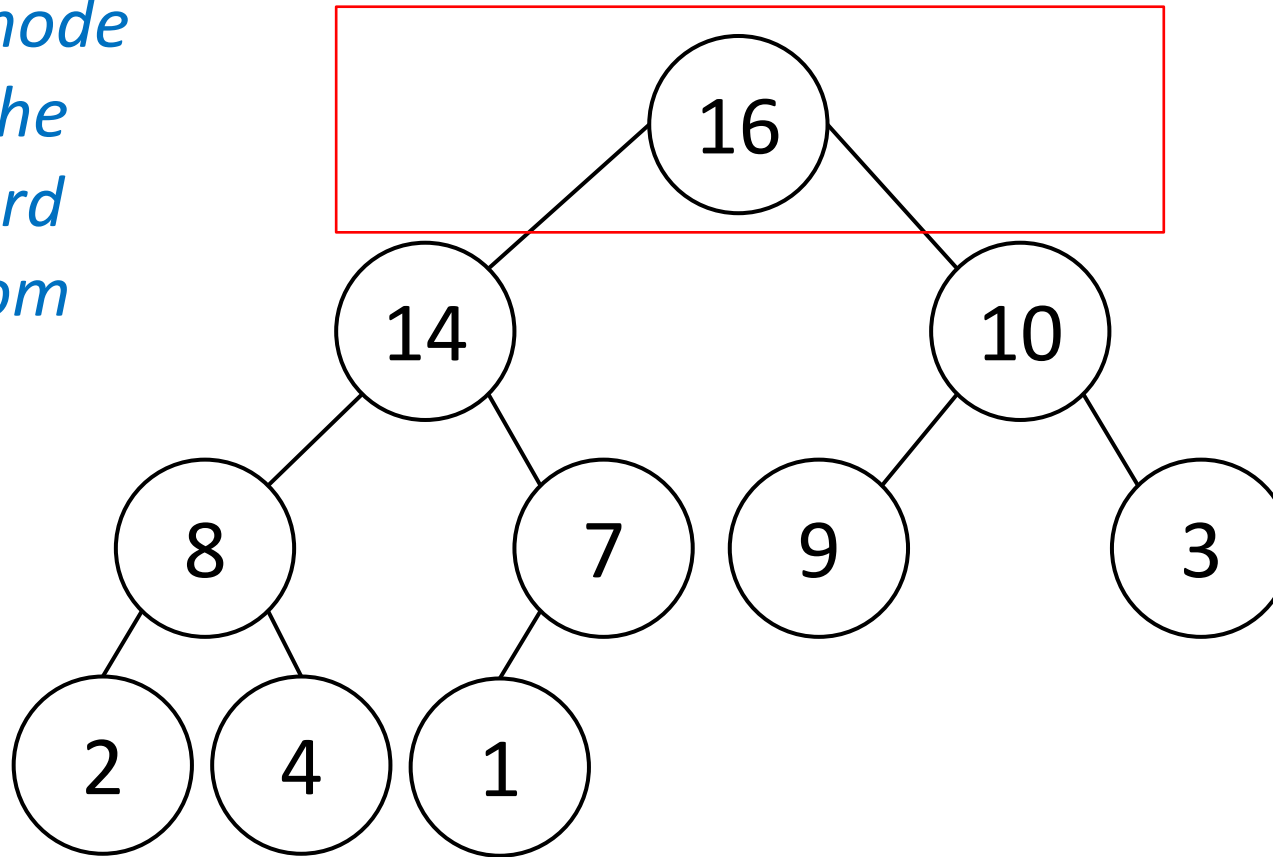
The **height** of a node is the length of the longest downward path to a leaf from that node



*height=2*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **height** of a node is the length of the longest downward path to a leaf from that node

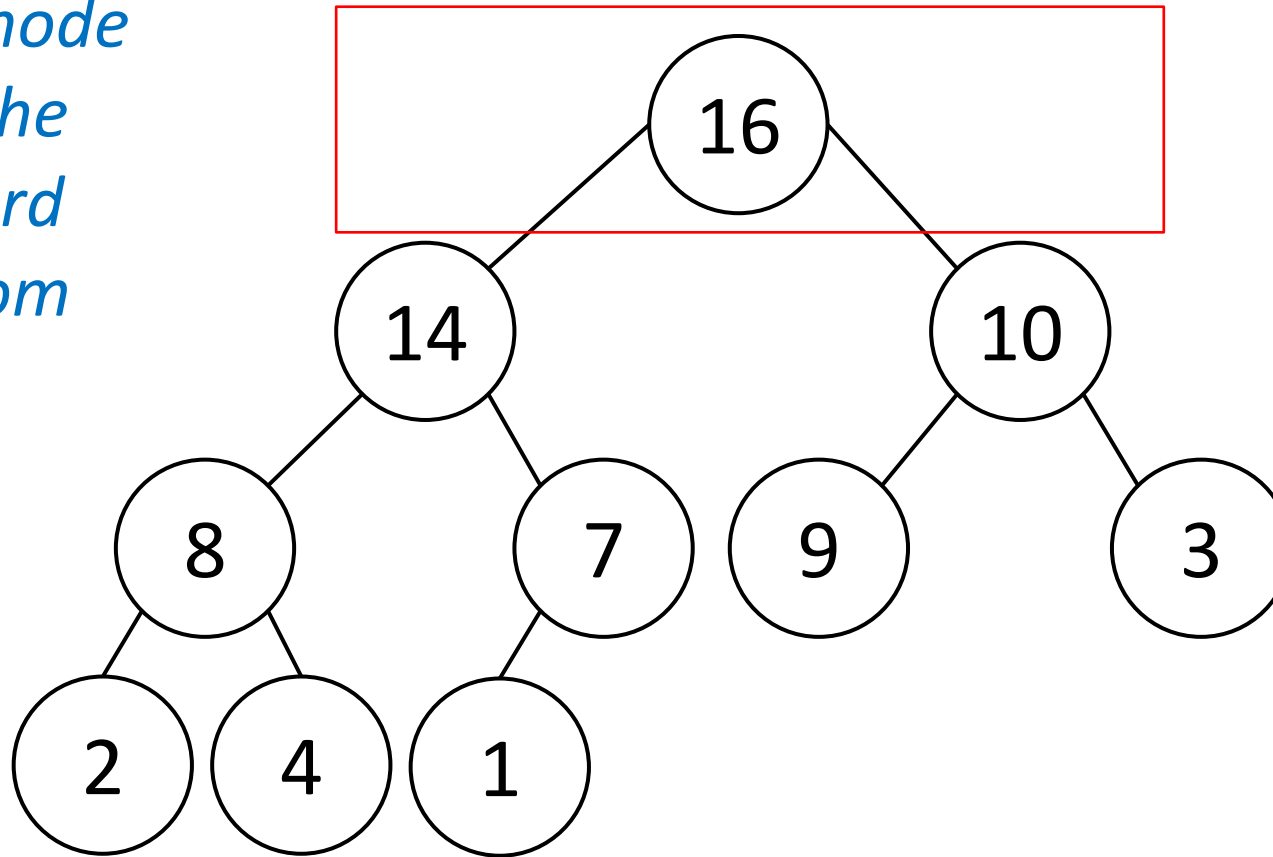


**height=3**

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |



The **height** of a node is the length of the longest downward path to a leaf from that node



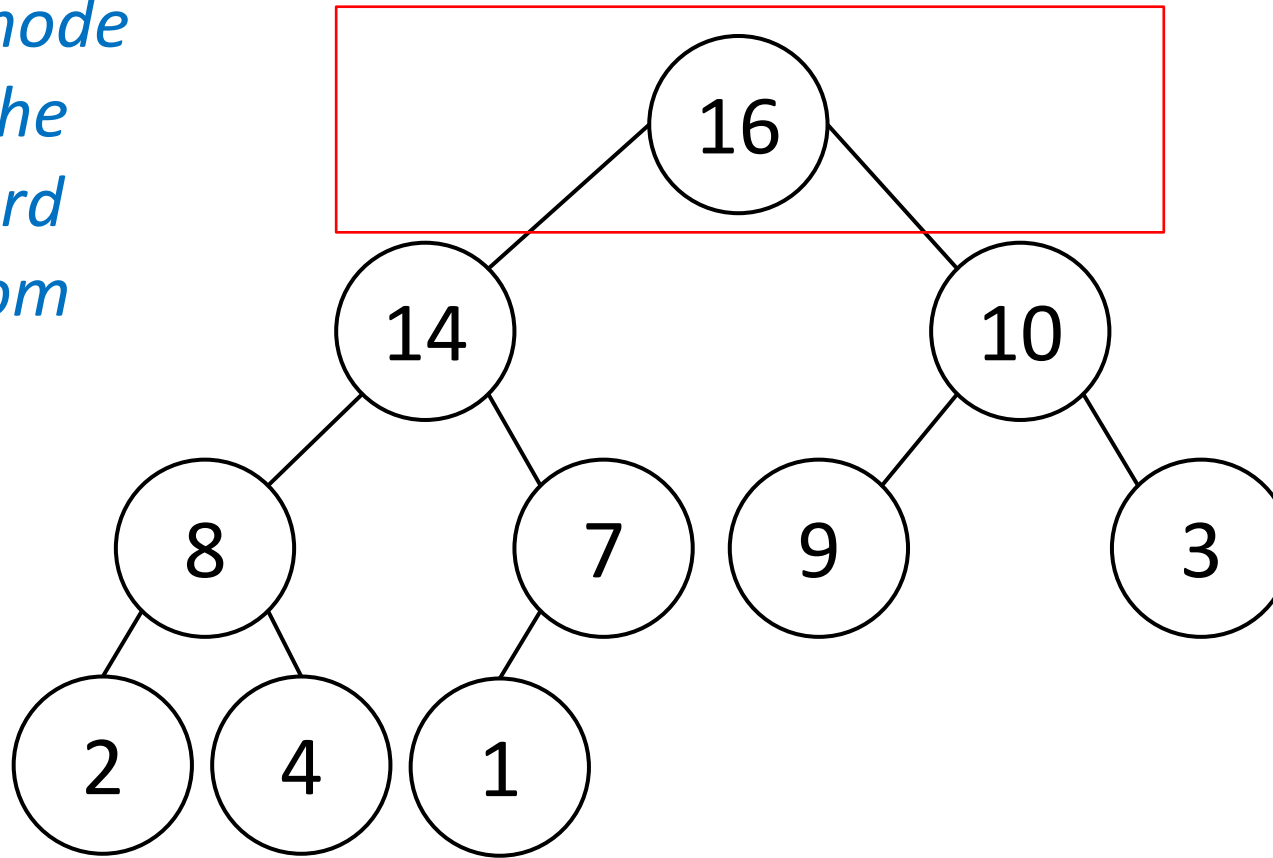
**height=3**

**height of tree**  
=  
**height of root**

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **height** of a node is the length of the longest downward path to a leaf from that node

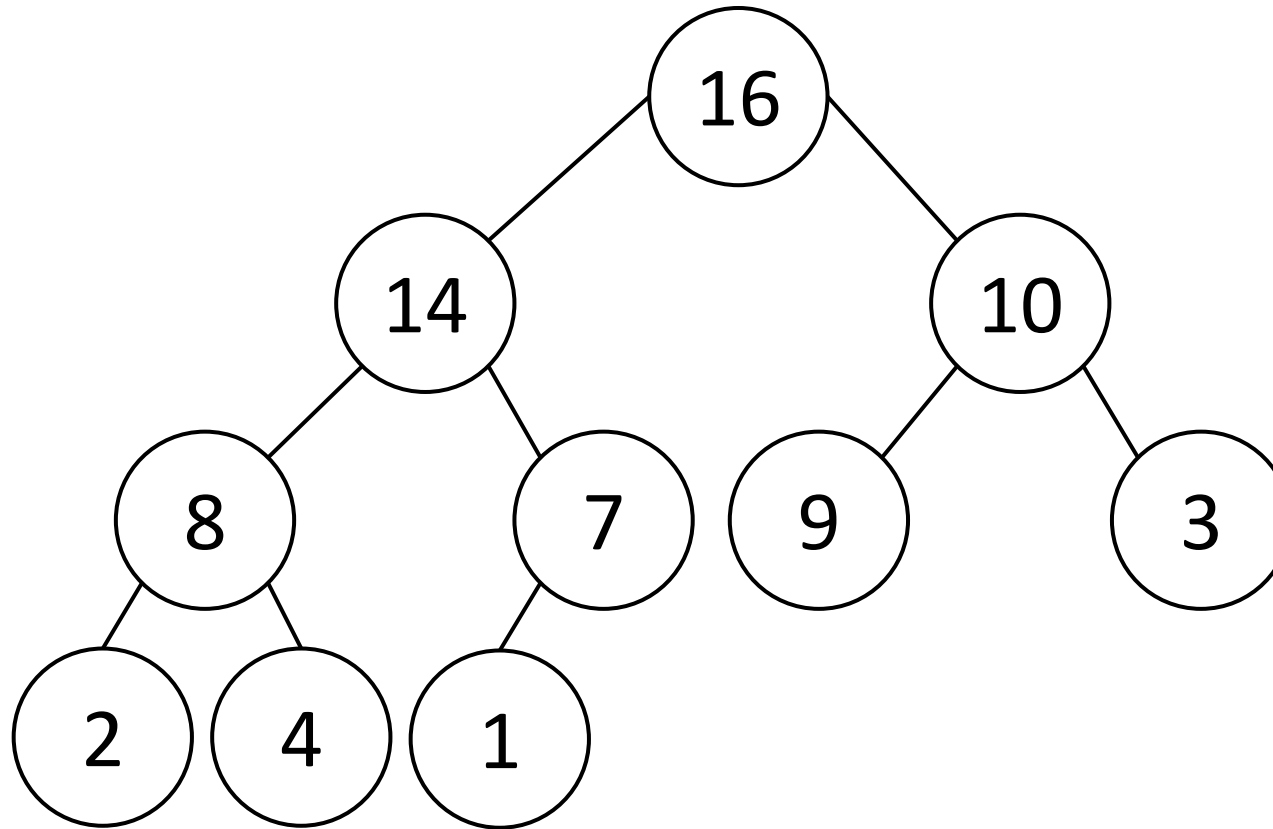
height of a complete binary tree  $O(\log N)$



height=3

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

*height of a  
complete  
binary tree  
 $O(\log N)$*

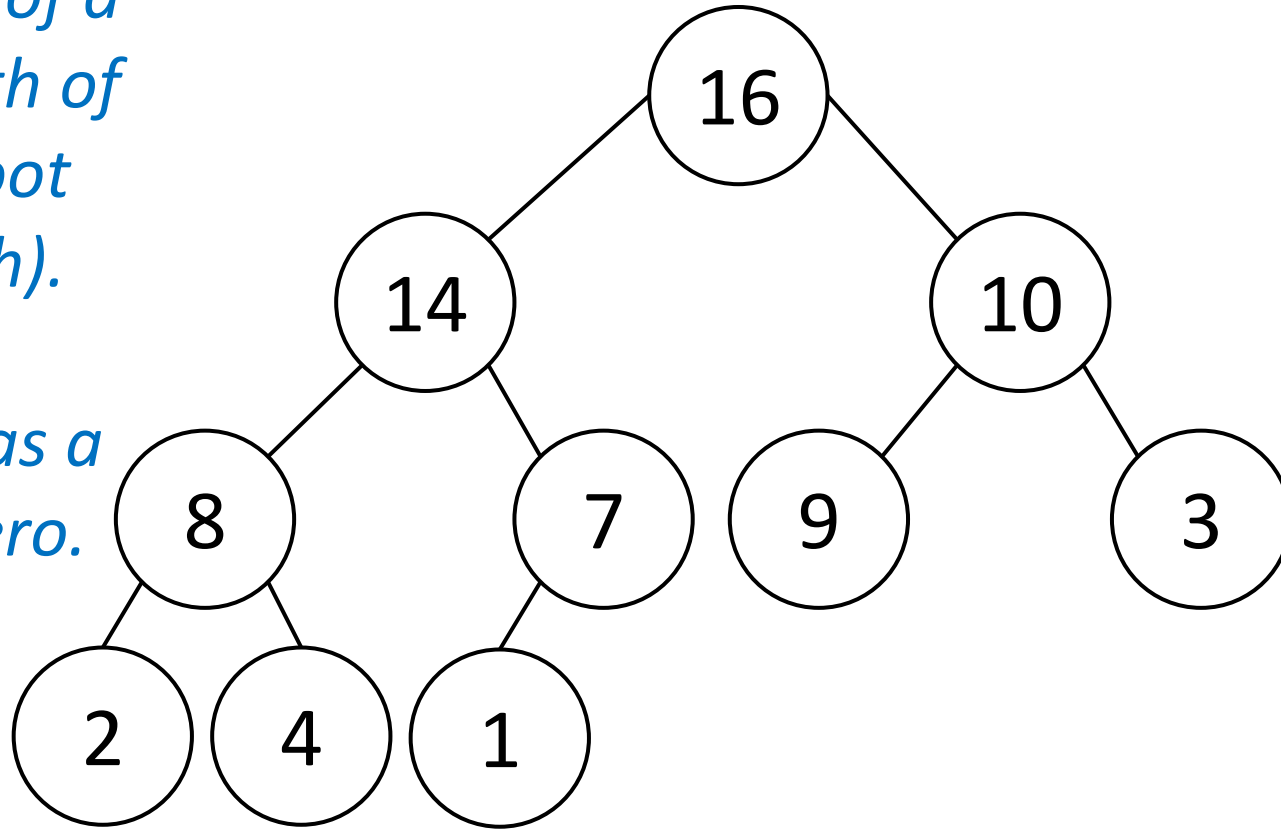


*Is height  
 $O(\log N)$  for  
all kinds of  
binary trees?*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **depth/level** of a node is the length of the path to its root (i.e., its root path).

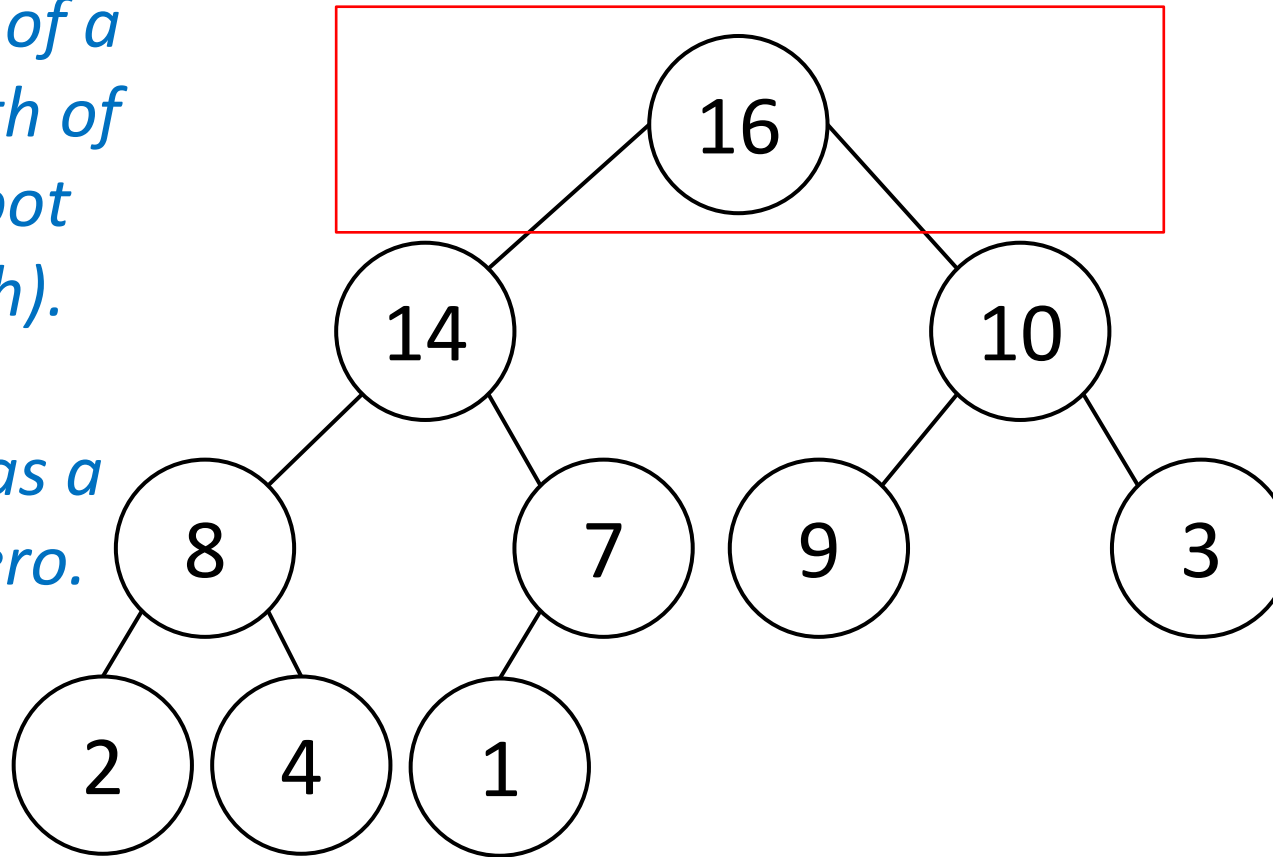
The root node has a depth/level of zero.



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **depth/level** of a node is the length of the path to its root (i.e., its root path).

The root node has a depth/level of zero.

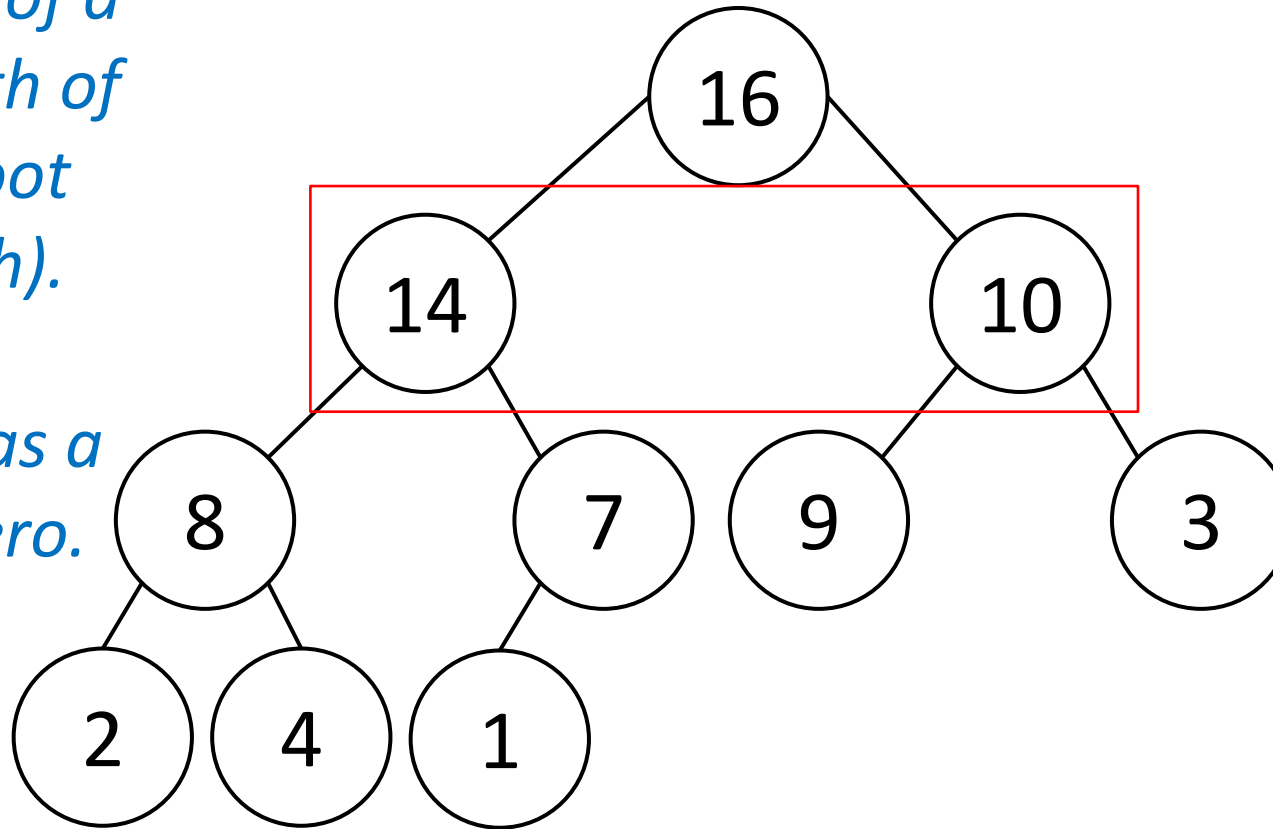


*depth/level=0*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **depth/level** of a node is the length of the path to its root (i.e., its root path).

The root node has a depth/level of zero.

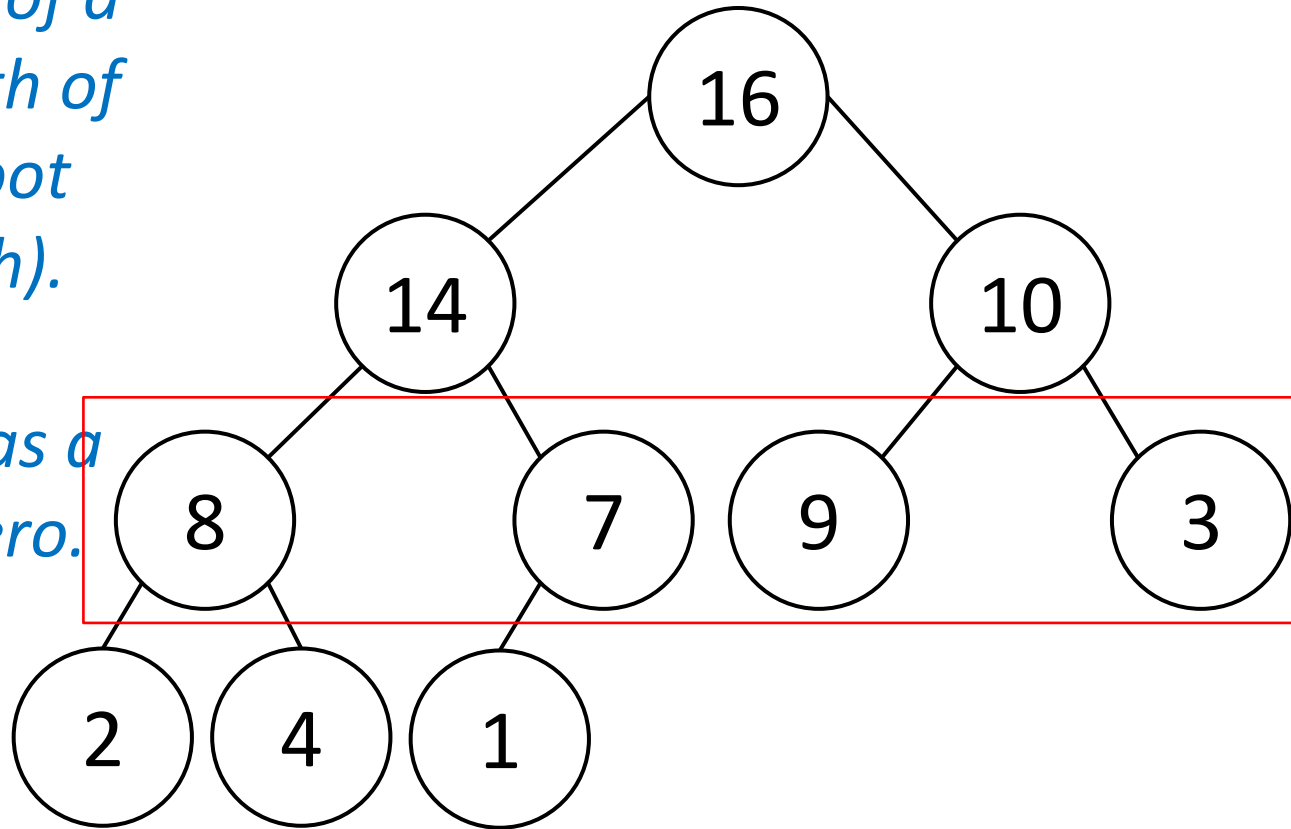


*depth/level=1*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **depth/level** of a node is the length of the path to its root (i.e., its root path).

The root node has a depth/level of zero.

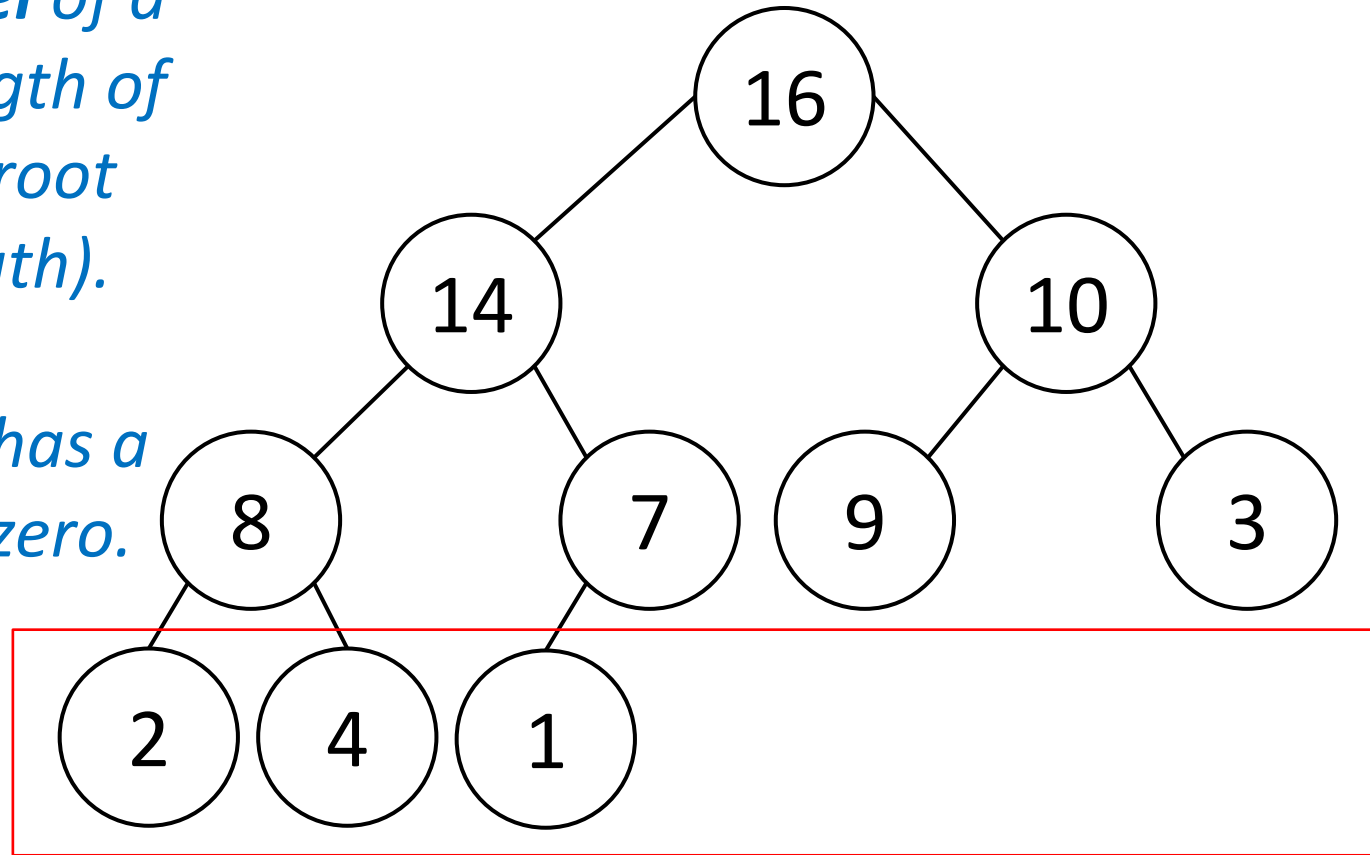


*depth/level=2*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

The **depth/level** of a node is the length of the path to its root (i.e., its root path).

The root node has a depth/level of zero.

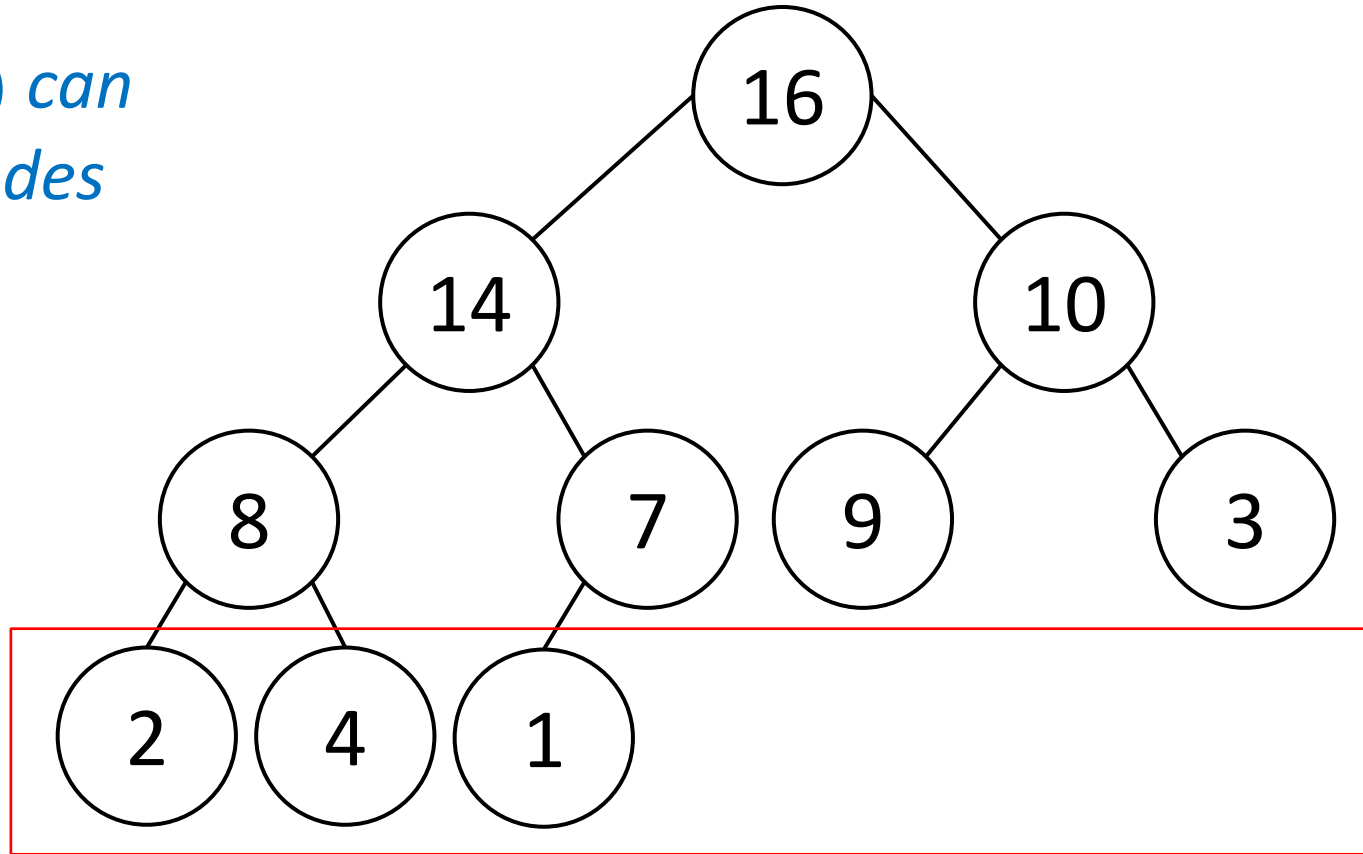


*depth/level=3*

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |



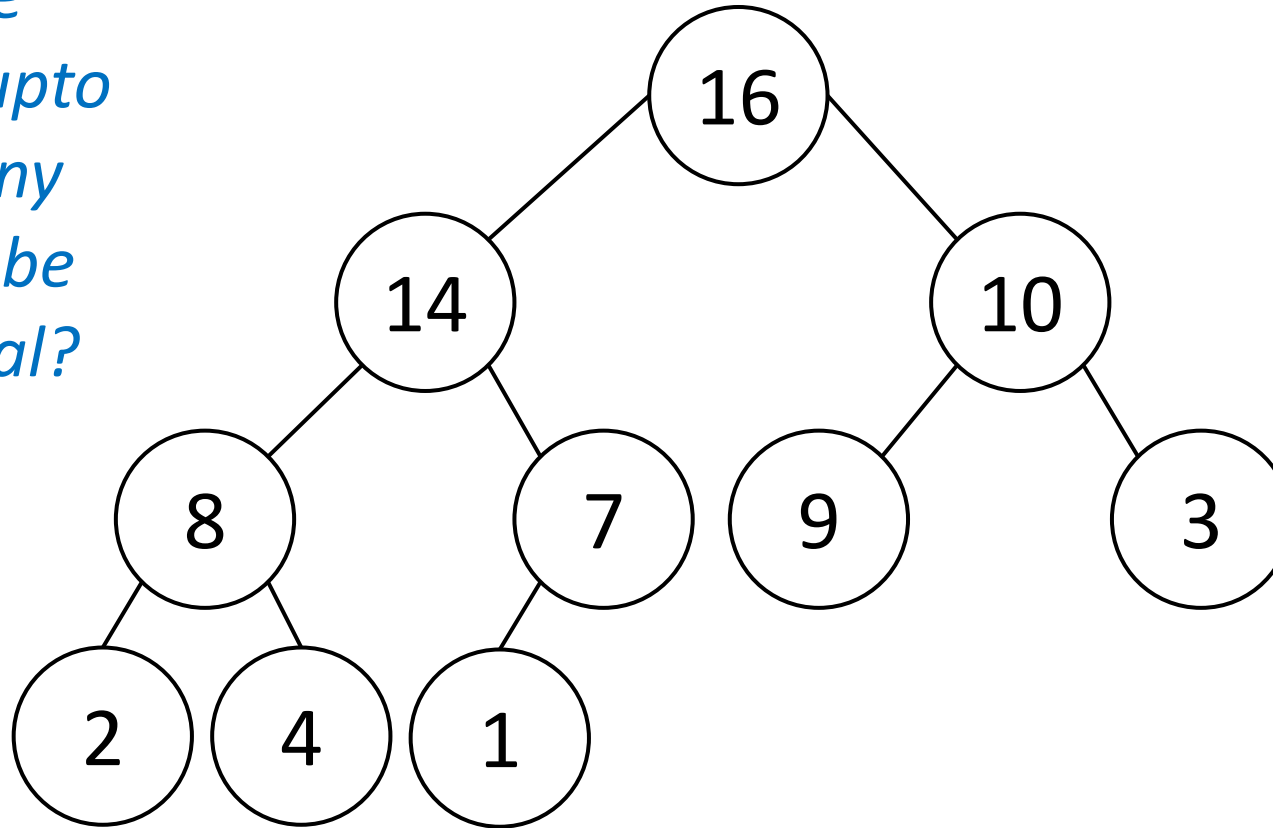
A particular  
**depth/level** ( $l$ ) can  
hold max  $2^l$  nodes



**Level 3:**  
 $2^3 = 8$  nodes

|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

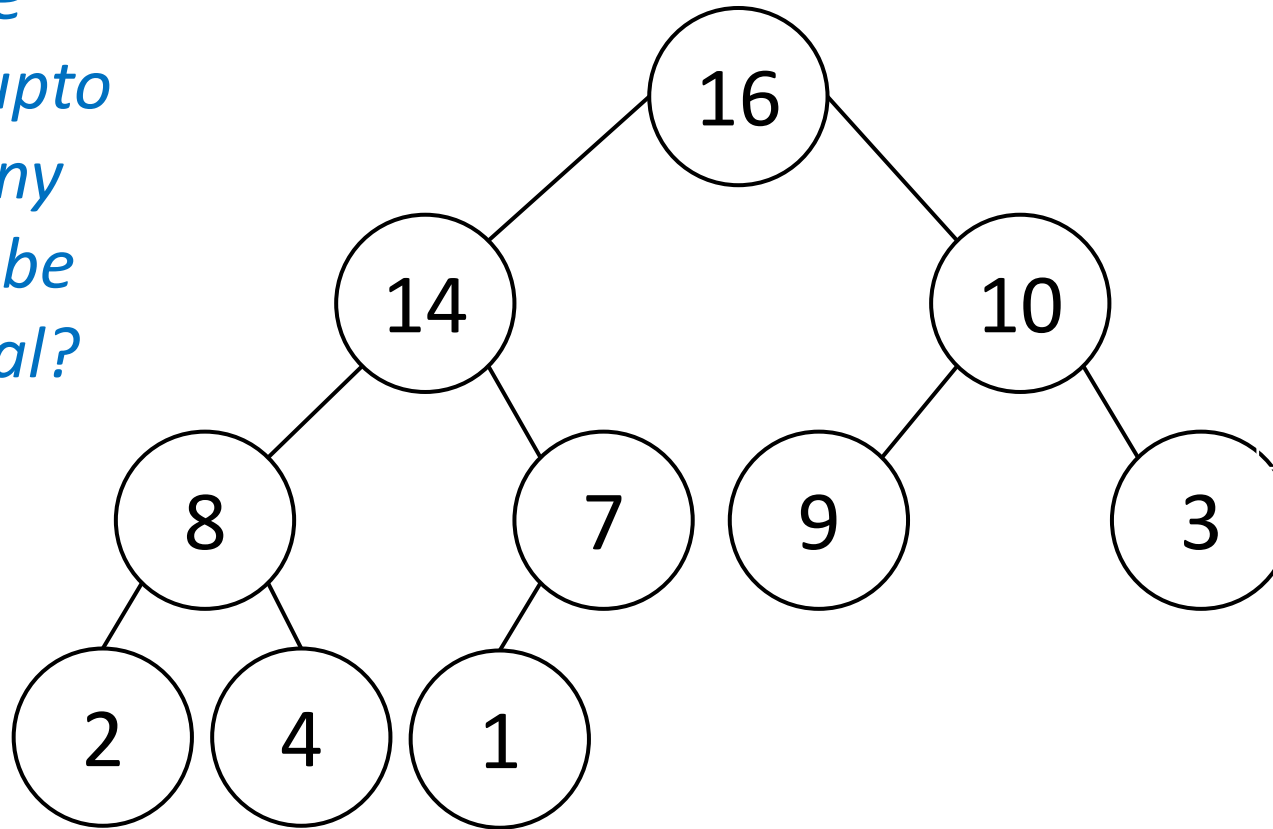
*Given a complete binary tree has upto  $l$  levels, how many nodes can there be in the tree in total?*



|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

Given a complete binary tree has upto  $l$  levels, how many nodes can there be in the tree in total?

$$2^{l+1} - 1$$



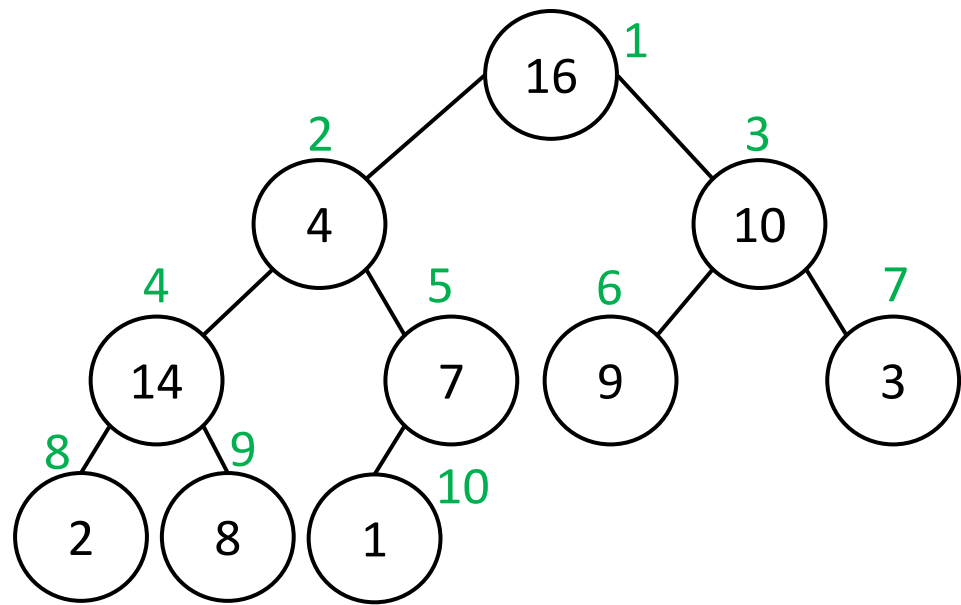
We have upto level-3; the tree can hold  $2^{3+1} - 1 = 15$  nodes in total

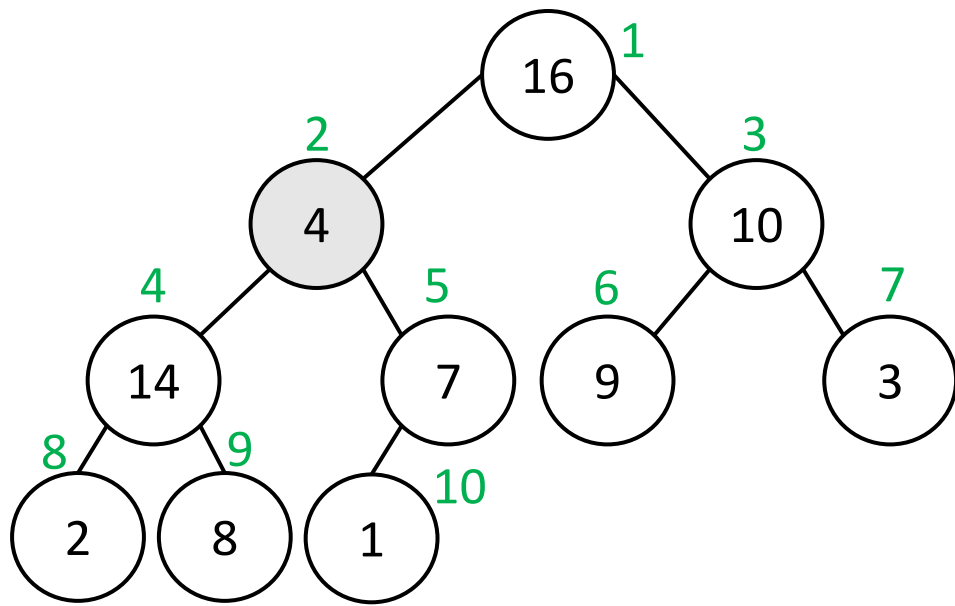
|    |    |    |   |   |   |   |   |   |    |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |
| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

*How to fix a node that violates the 'heap' condition?*

*How to fix a node that violates the 'heap' condition?*

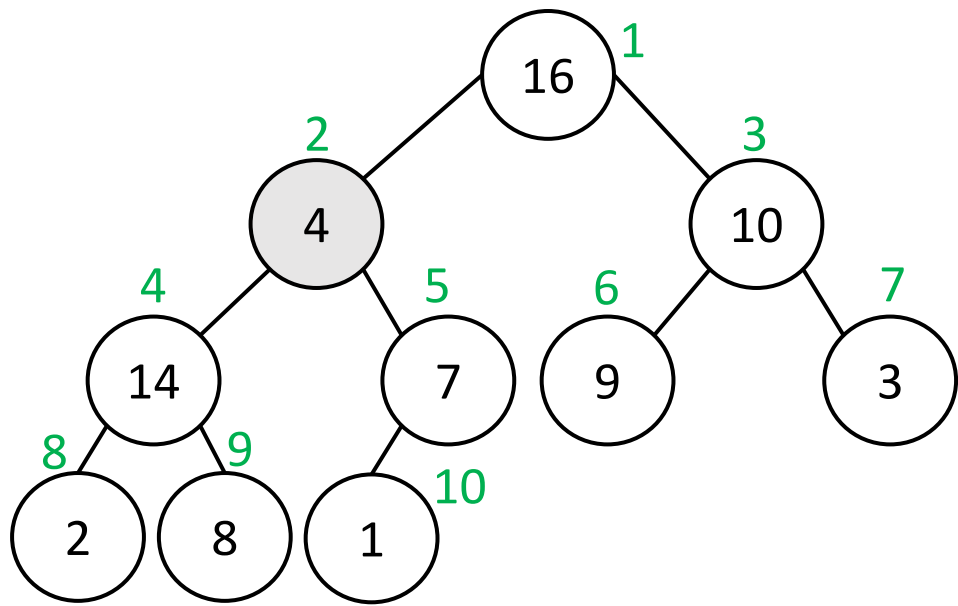
*heapify( $i^{th}$  node)*





*All nodes follow the  
maxheap condition  
except node#2*

*How to fix?*

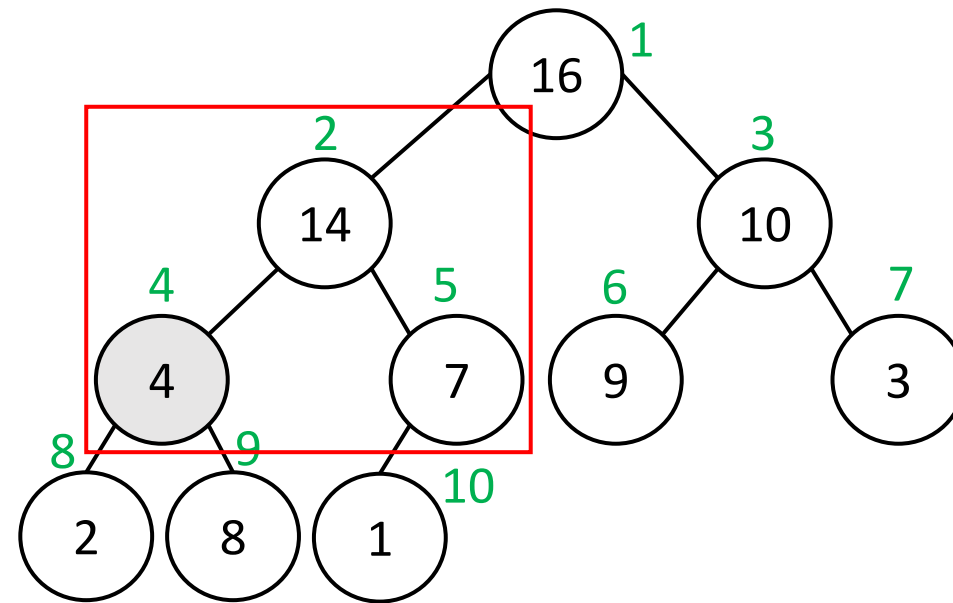
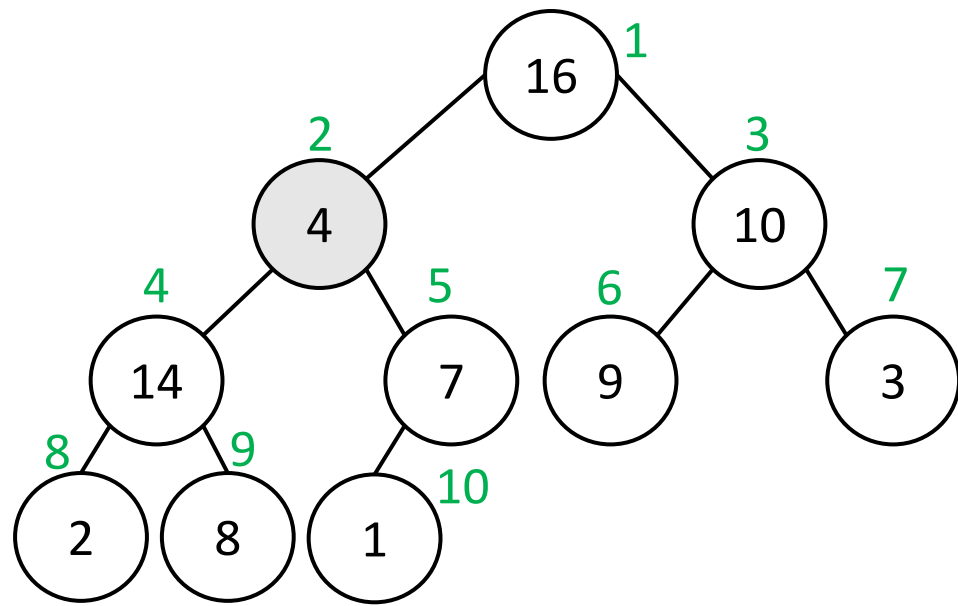


*All nodes follow the  
maxheap condition  
except node#2*

*How to fix?*

*heapify(node#2)*

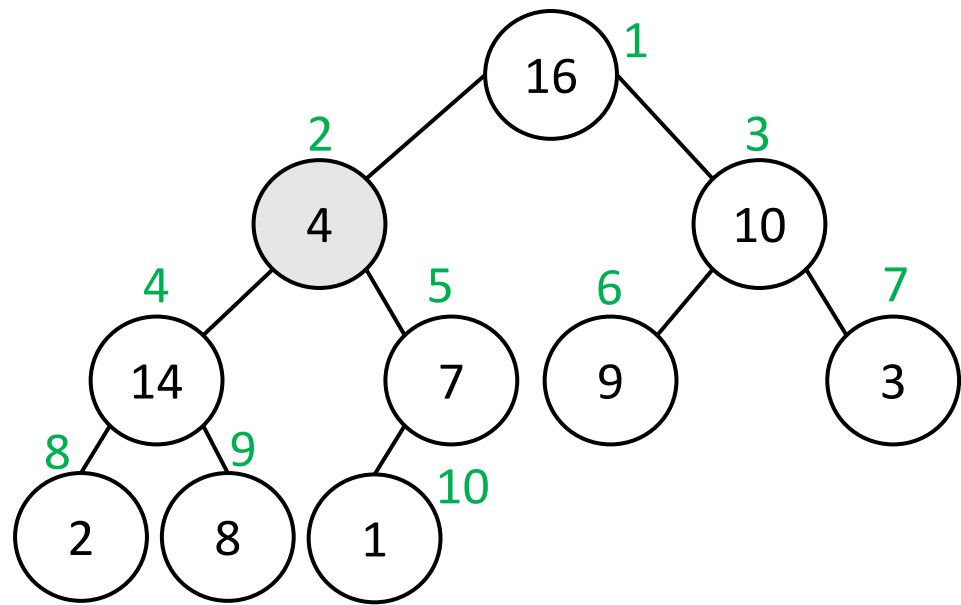




*All nodes follow the  
maxheap condition  
except node#2*

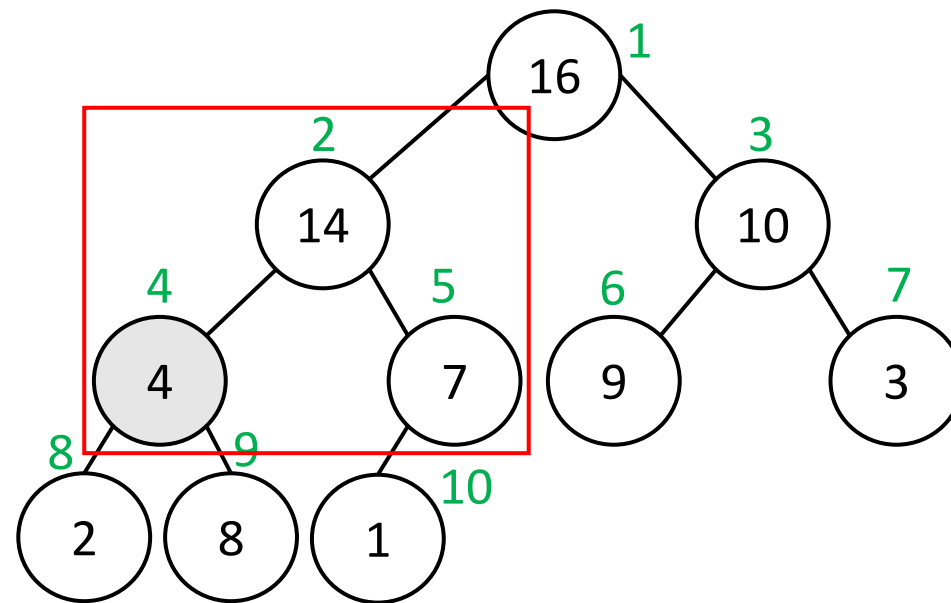
*How to fix?*

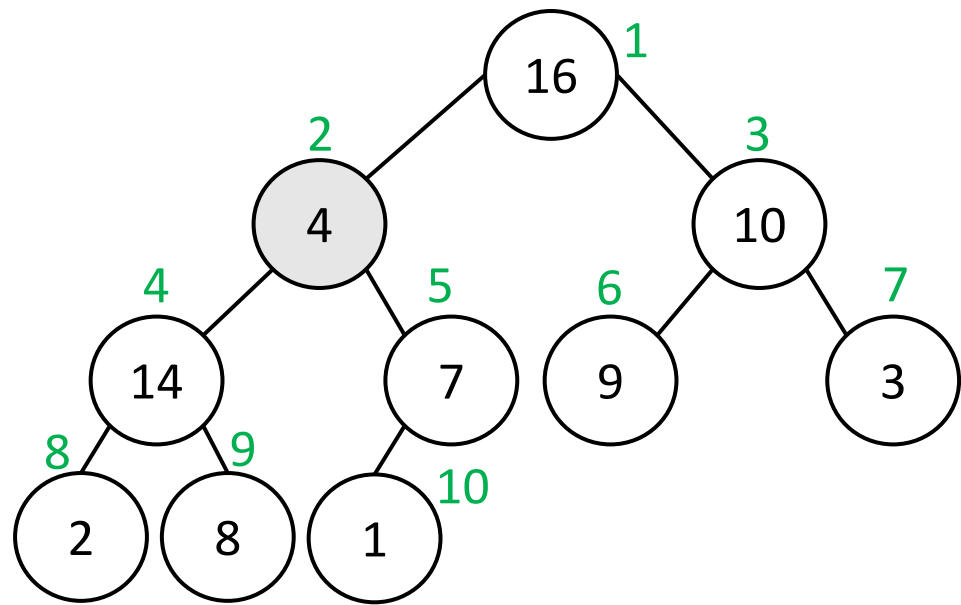
*heapify(node#2)*



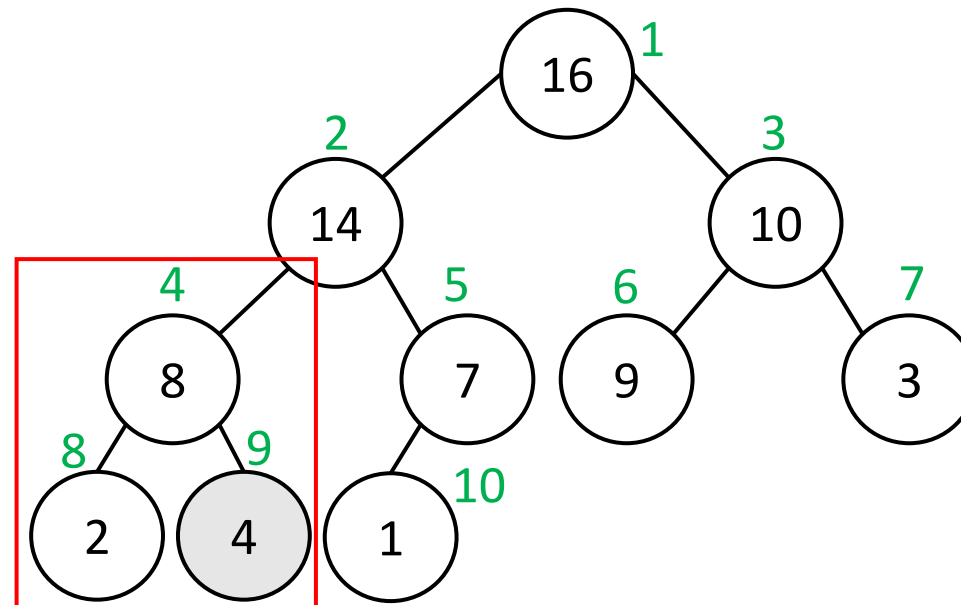
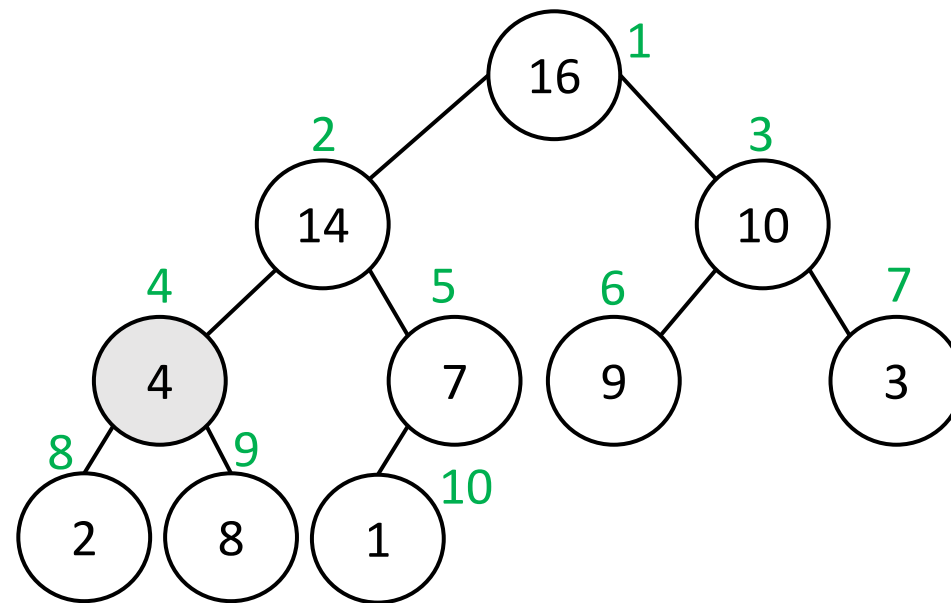
*Can't stop here!*

*heapify(node#4)*





*All nodes now satisfy the maxHeap condition.*



```
max_heapify(i){
```

```
}
```

*We need to heapify the  
 $i^{th}$  node*

```
max_heapify(i){
```

Idea:

Need to compare with the left and right child  
Whichever is bigger, swap with that!

```
}
```

```
max_heapify(i){  
    l=2i  
    r=2i+1  
  
}
```

```
max_heapify(i){  
    l=2i  
    r=2i+1  
    if l <= heapSize and heap[l] > heap[i]  
        largest=l  
    else  
        largest=i  
  
}
```

```
max_heapify(i){  
    l=2i  
    r=2i+1  
    if l <= heapSize and heap[l] > heap[i]  
        largest=l  
    else  
        largest=i  
    if r <= heapSize and heap[r] > heap[largest]  
        largest=r  
  
}
```



```
max_heapify(i){  
    l=2i  
    r=2i+1  
    if l <= heapSize and heap[l] > heap[i]  
        largest=l  
    else  
        largest=i  
    if r <= heapSize and heap[r] > heap[largest]  
        largest=r  
    if largest != i  
        exchange heap[i] with heap[largest]  
  
}
```

```
max_heapify(i){  
    l=2i  
    r=2i+1  
    if l <= heapSize and heap[l] > heap[i]  
        largest=l  
    else  
        largest=i  
    if r <= heapSize and heap[r] > heap[largest]  
        largest=r  
    if largest != i  
        exchange heap[i] with heap[largest]  
}
```

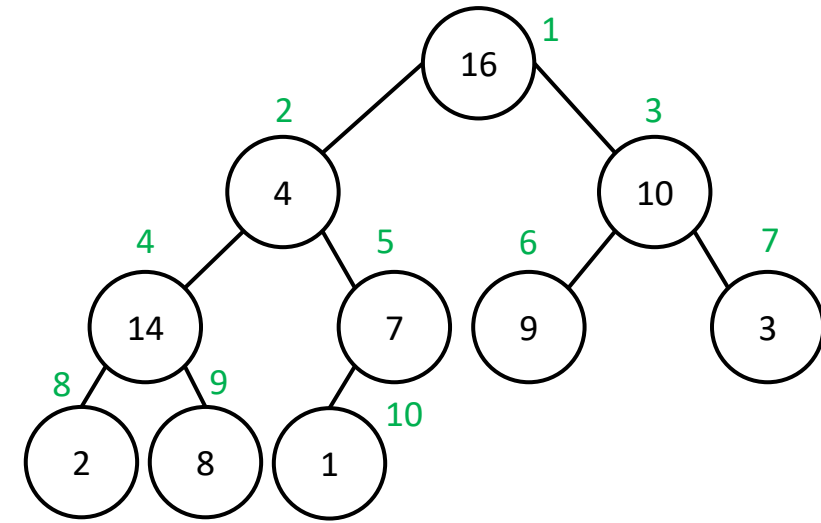
*Done?*

```
max_heapify(i){  
    l=2i  
    r=2i+1  
    if l <= heapSize and heap[l] > heap[i]  
        largest=l  
    else  
        largest=i  
    if r <= heapSize and heap[r] > heap[largest]  
        largest=r  
    if largest != i  
        exchange heap[i] with heap[largest]  
        max_heapify(largest)  
}
```

```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
        max_heapify(largest)
}

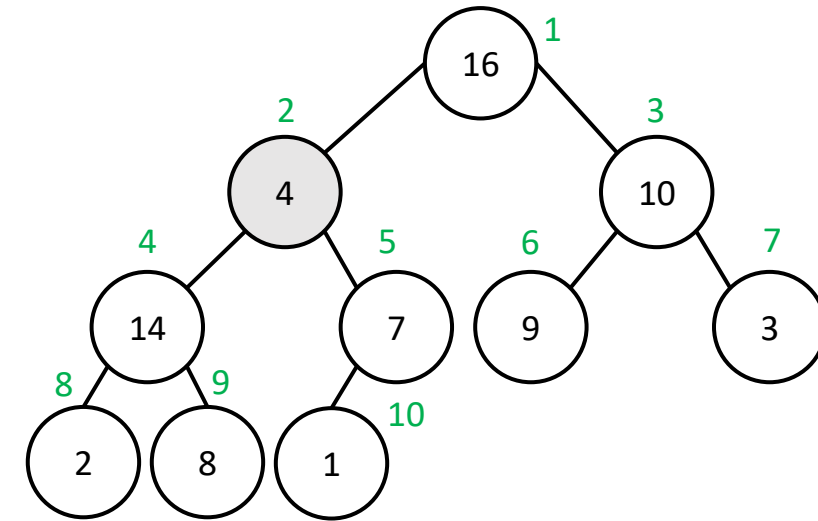
```



```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
        max_heapify(largest)
}

```



```

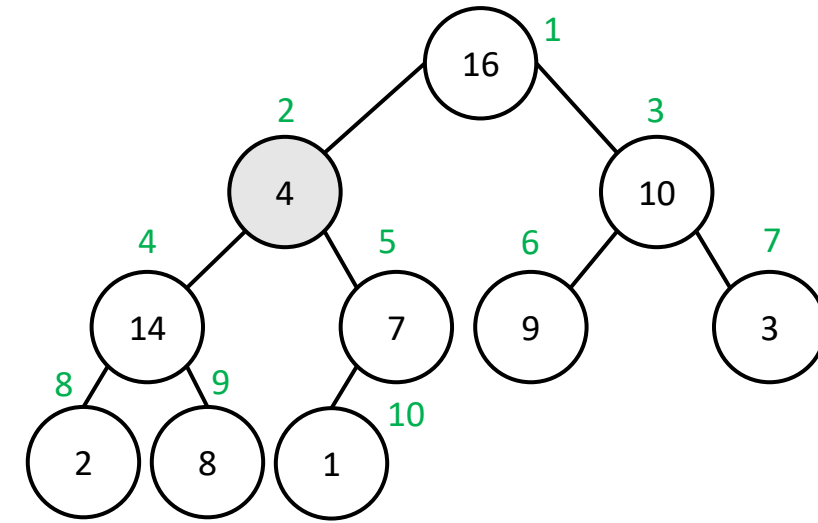
maxHeapify(2)
l=4, r=5
largest=4
swap(heap[2], heap[4])
maxHeapify(4)

```

```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
        max_heapify(largest)
}

```



```

maxHeapify(2)
l=4, r=5
largest=4
swap(heap[2], heap[4])
maxHeapify(4)

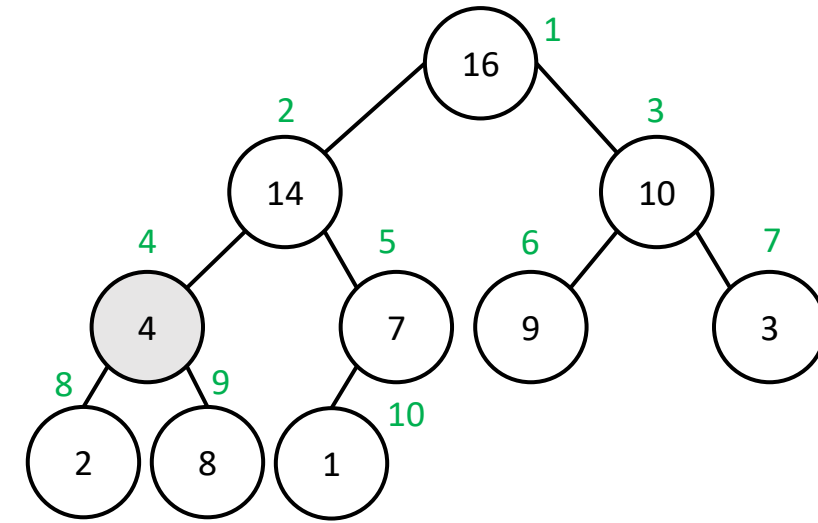
```

*Note: compare the  
contents, not index!*

```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
        max_heapify(largest)
}

```



```

maxHeapify(2)
l=4, r=5
largest=4
swap(heap[2], heap[4])
maxHeapify(4)

```

```

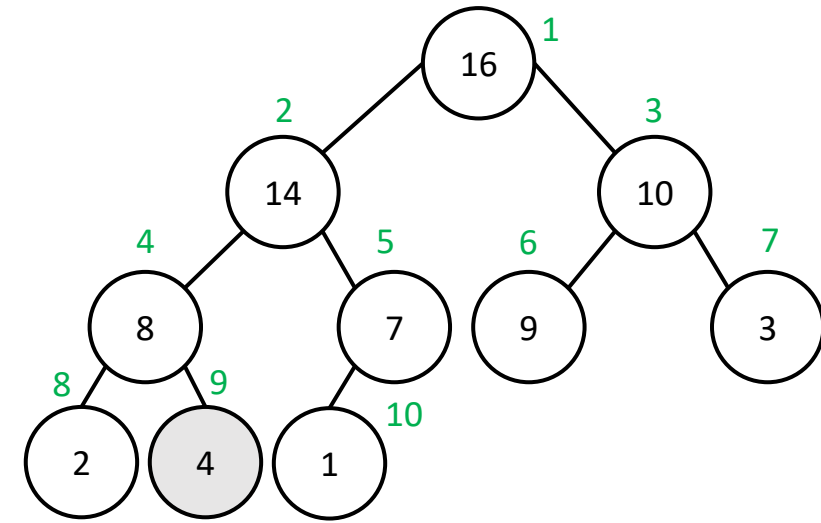
maxHeapify(4)
l=8, r=9
largest=9
swap(heap[4], heap[9])
maxHeapify(9)

```

```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
        max_heapify(largest)
}

```



```

maxHeapify(2)
l=4, r=5
largest=4
swap(heap[2], heap[4])
maxHeapify(4)

```

```

maxHeapify(4)
l=8, r=9
largest=9
swap(heap[4], heap[9])
maxHeapify(9)

```

```

maxHeapify(9)
l=18, r=19
l>heapsize
r>heapsize
i=9 is largest

```

So, it doesn't go  
inside if condition.  
Break!



```
max_heapify(i){  
    l=2i  
    r=2i+1  
    if l <= heapSize and heap[l] > heap[i]  
        largest=l  
    else  
        largest=i  
    if r <= heapSize and heap[r] > heap[largest]  
        largest=r  
    if largest != i  
        exchange heap[i] with heap[largest]  
        max_heapify(largest)  
}
```

*What is the time complexity  
of heapify?*

```
max_heapify(i){
```

```
    l=2i
```

```
    r=2i+1
```

```
    if l <= heapSize and heap[l] > heap[i]
```

```
        largest=l
```

```
    else
```

```
        largest=i
```

```
    if r <= heapSize and heap[r] > heap[largest]
```

```
        largest=r
```

```
    if largest != i
```

```
        exchange heap[i] with heap[largest]
```

```
        max_heapify(largest)
```

```
}
```

*O(1)*

```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
    max_heapify(largest)
}

```

*Time complexity for recursive functions is defined by the number of times the function calls itself.*

```

max_heapify(i){
    l=2i
    r=2i+1
    if l <= heapSize and heap[l] > heap[i]
        largest=l
    else
        largest=i
    if r <= heapSize and heap[r] > heap[largest]
        largest=r
    if largest != i
        exchange heap[i] with heap[largest]
        max_heapify(largest)
}

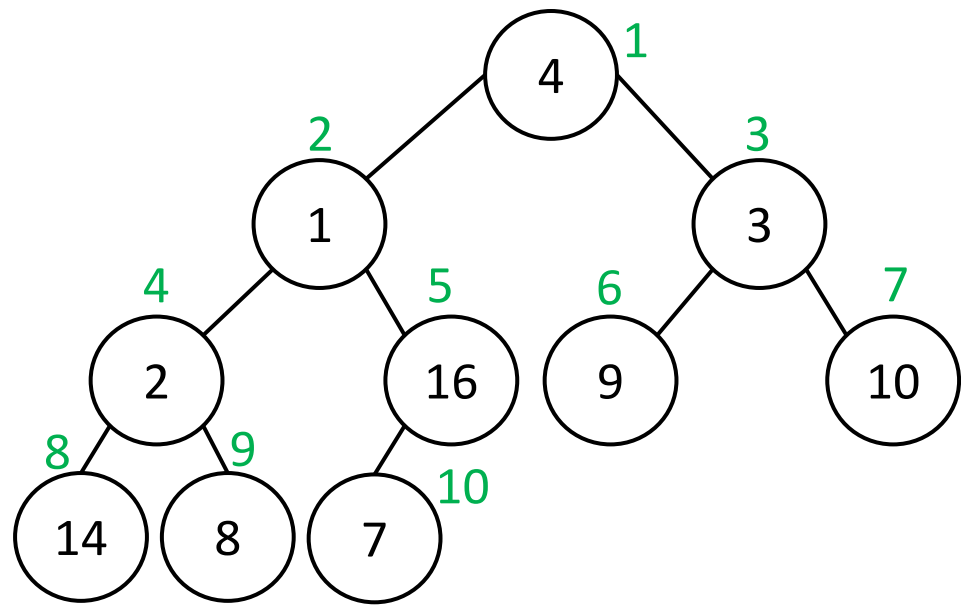
```

*In this case, the  
maximum #of  
recursive calls =  
height of the tree  
=  $O(h)$   
 $\sim O(\log N)$*

*Time complexity for recursive functions is defined by the  
number of times the function calls itself.*

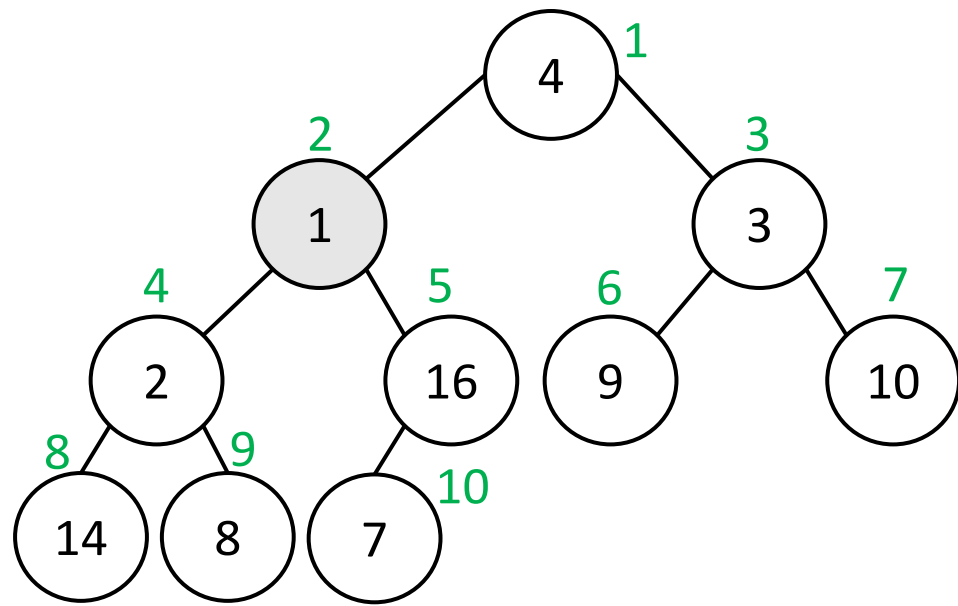
*Heapify only fixes one location.*

*Need to ensure each that node satisfies  
the heap condition.*



*Suppose, we want to build a max-heap using this set of numbers*

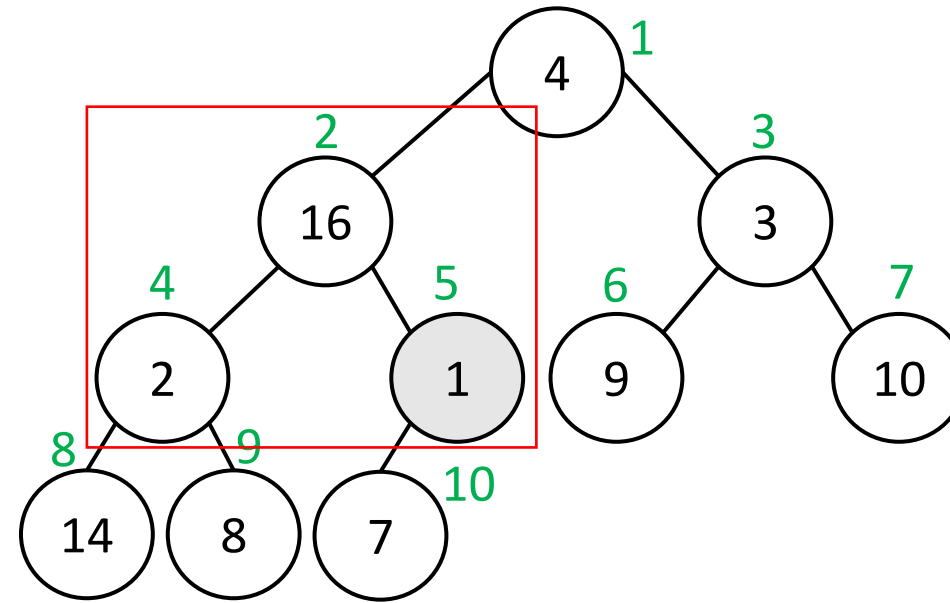
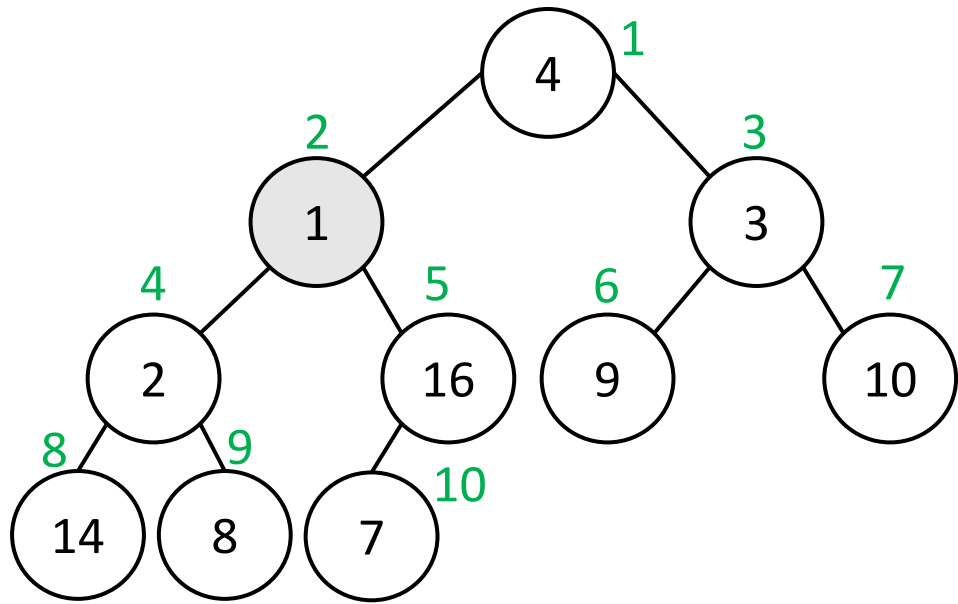
*Let's start from the root and heapify each node.*



*Let's start from the root  
and heapify each node.*

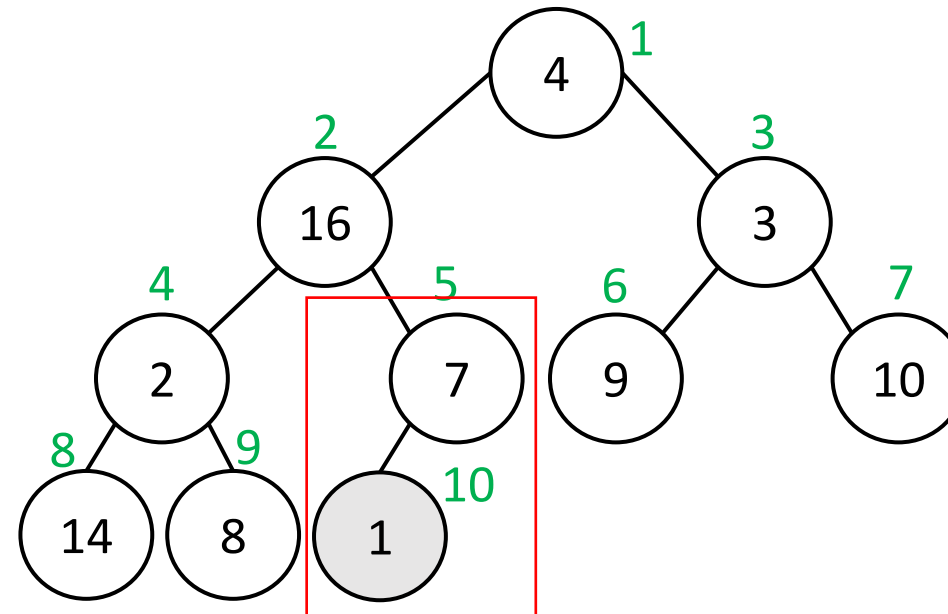
*Node#1, okay*

*Node#2, violation!  
heapify(2)*

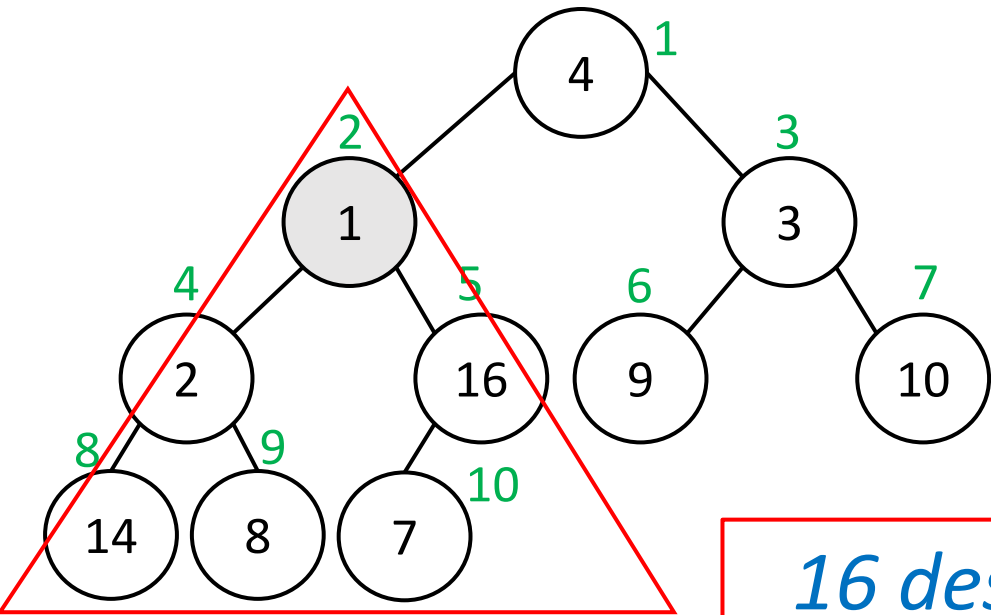


*Node#2 is fixed.*

*Do you notice any problem?*



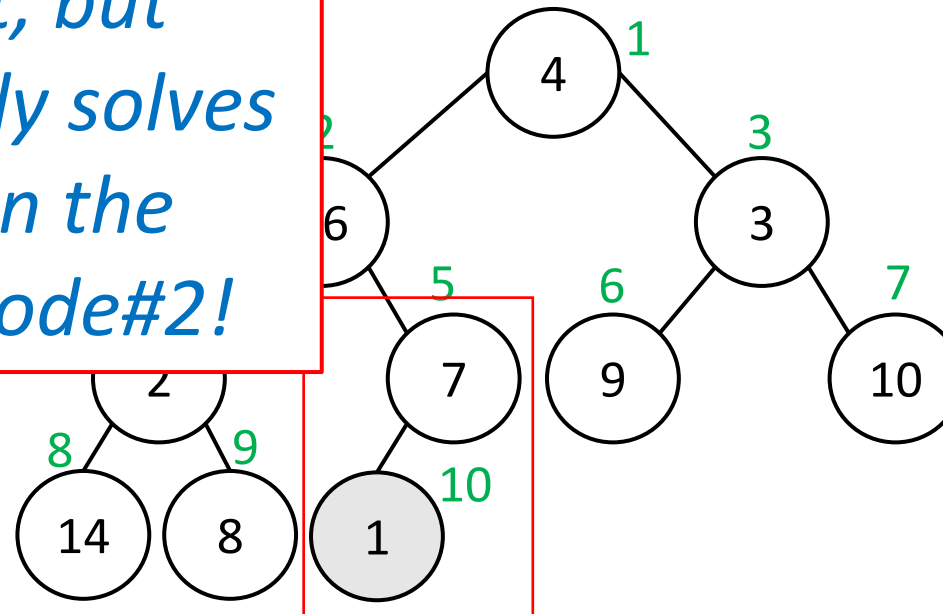
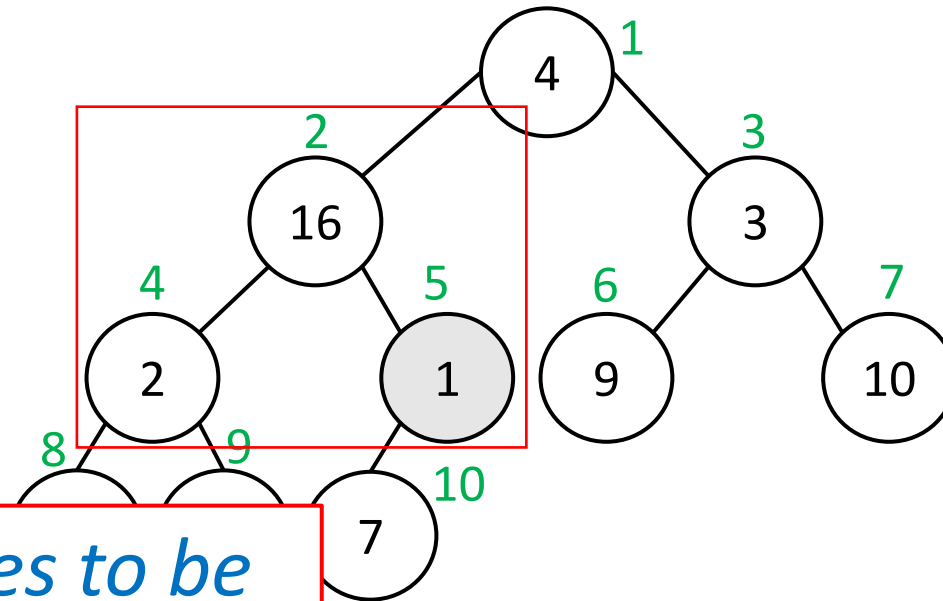




*Node#2 is fixed.*

*Do you notice any problem?*

*16 deserves to be the root, but heapify only solves what's in the subtree node#2!*



*Top-down approach isn't working here,  
need to go bottom-up!*

*Starting from the last node, need to go backward and heapify each node!*

*Starting from the last node, need to go backward and heapify each node!*

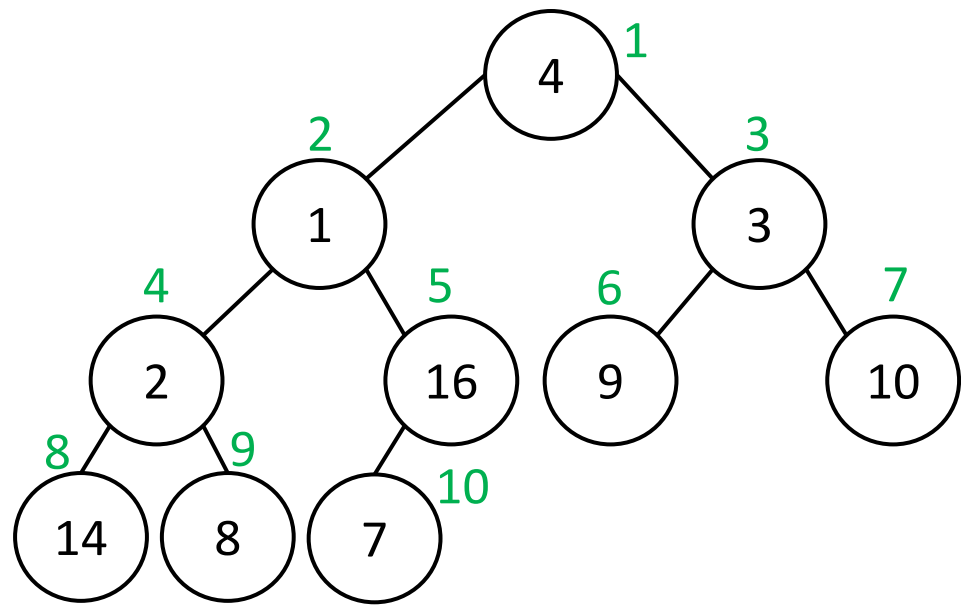
*Can we optimize this?*

*Starting from the last node, need to go backward and heapify each node!*

*Can we optimize this?*

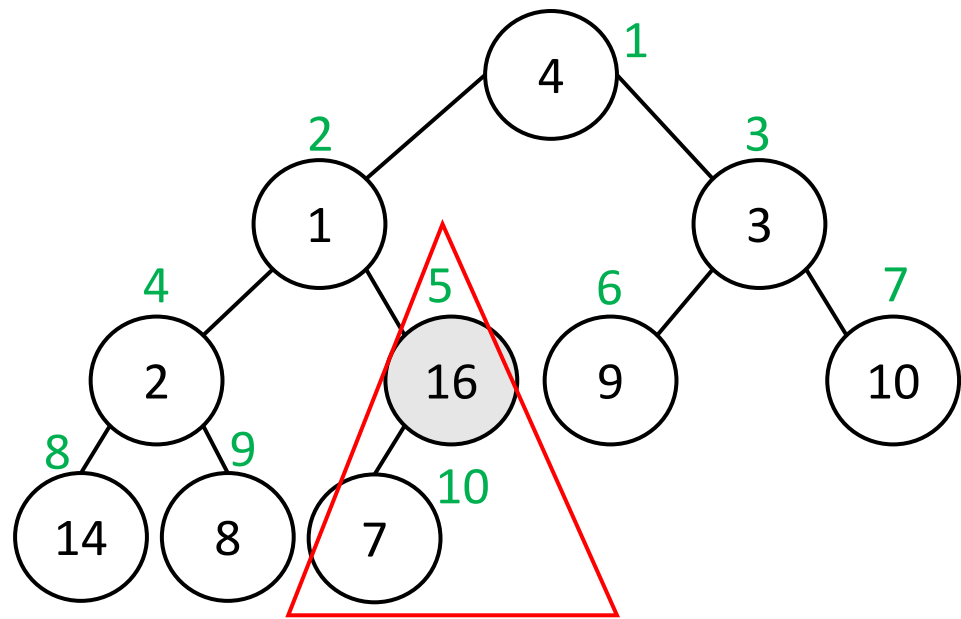
*We can start to heapify from the middle of the tree and go back to the root!*

*(After  $\frac{n}{2}$  th location, all nodes are leaf node)*



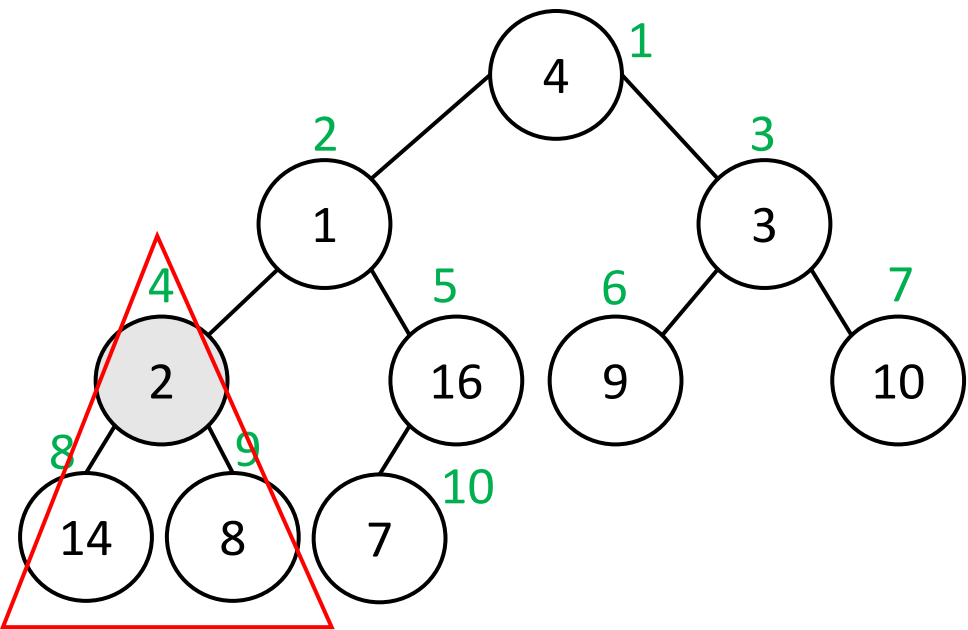
heapSize = 10

So start from index-5



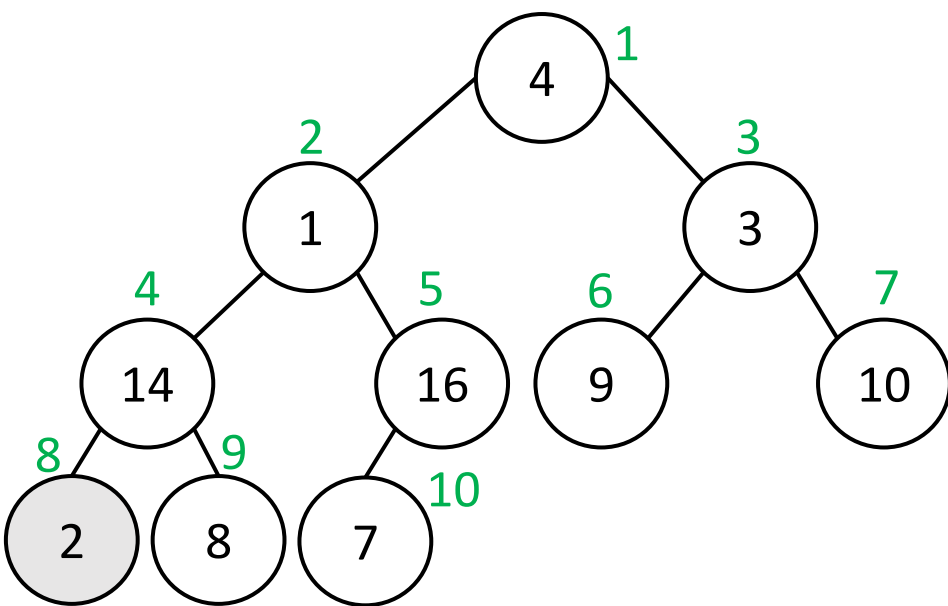
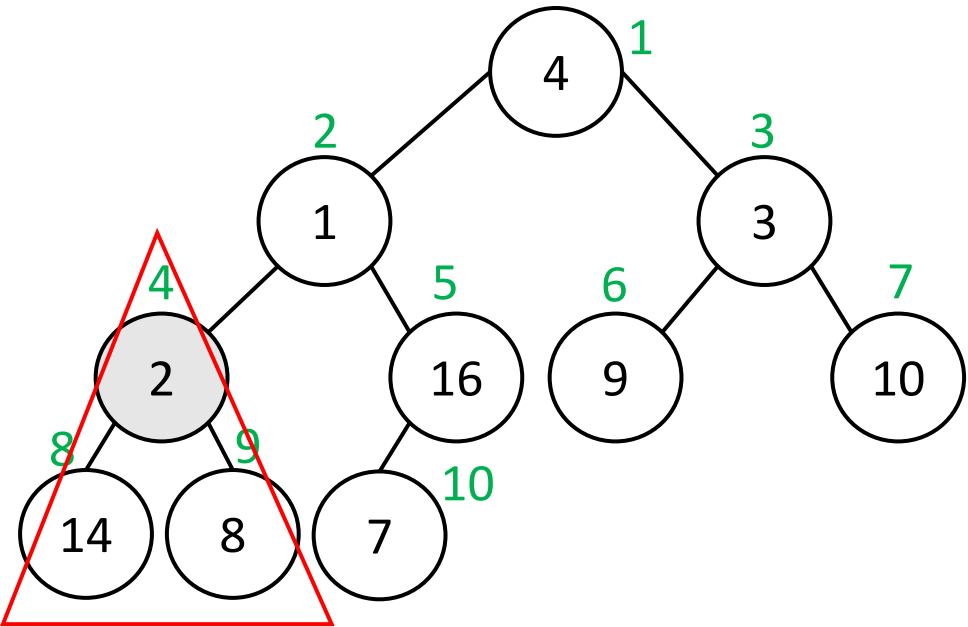
heapify(5)

No change needed

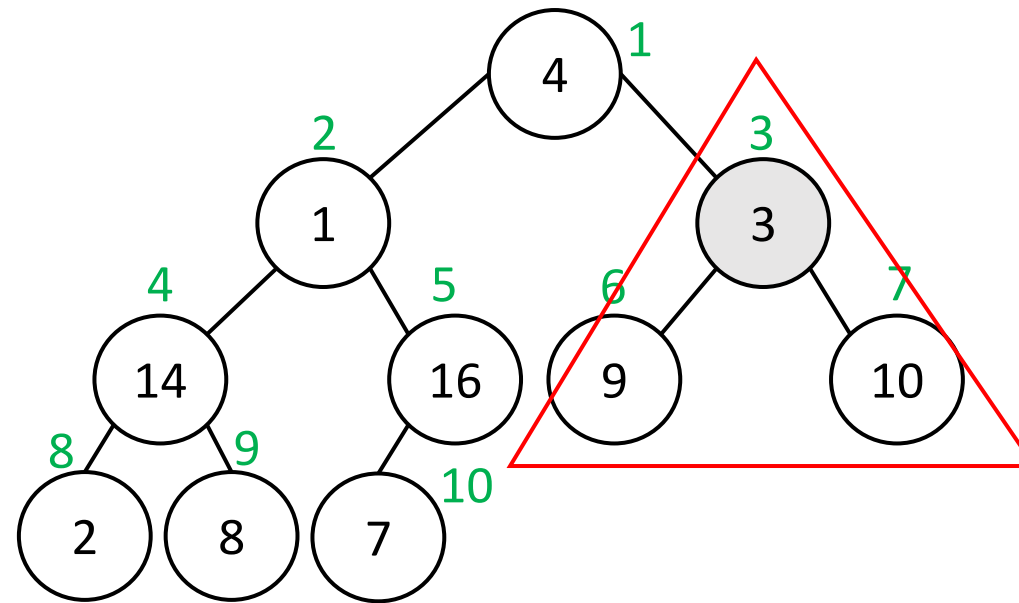


heapify(4)

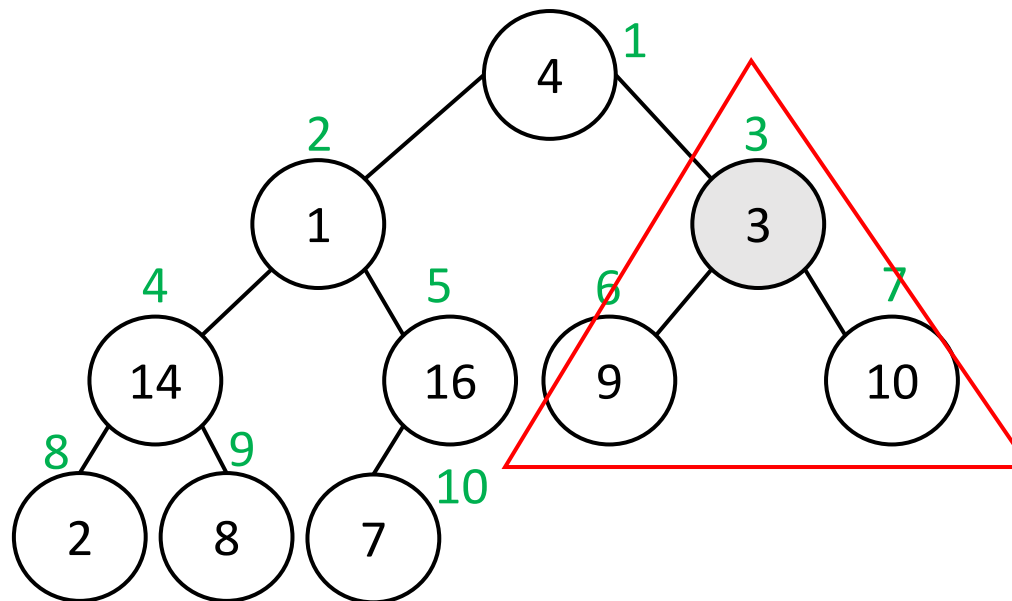




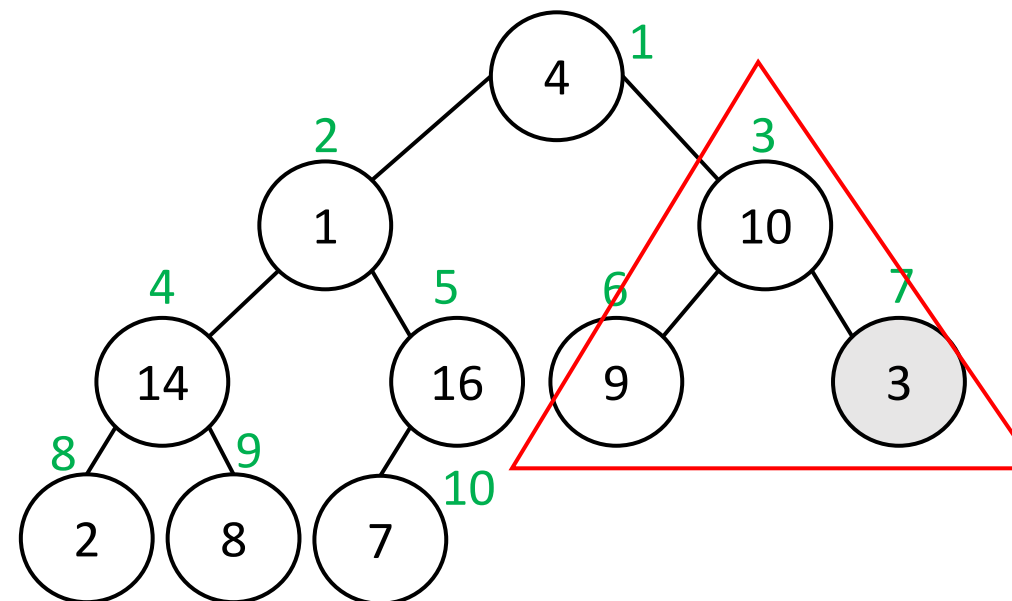
heapify(4)



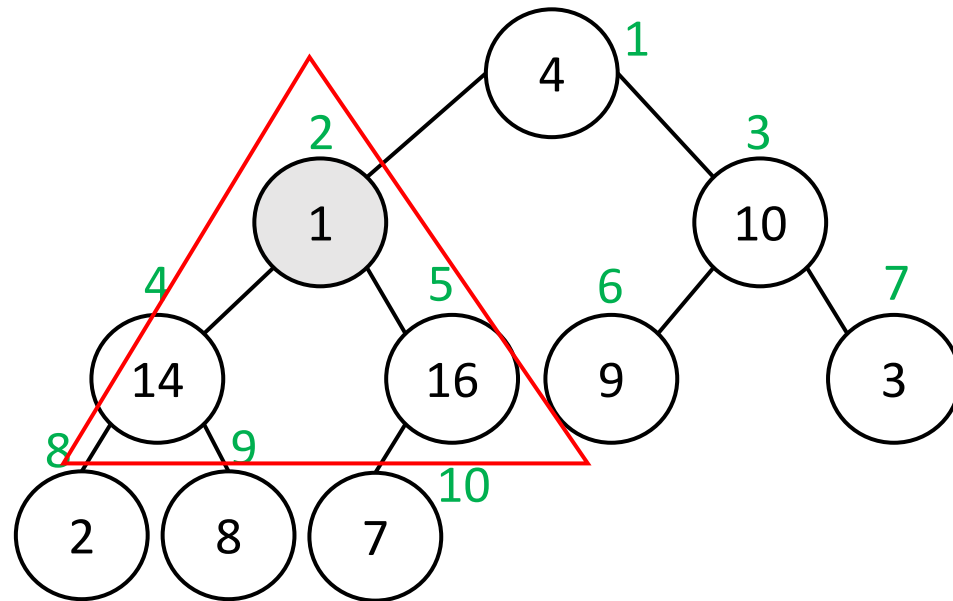
heapify(3)



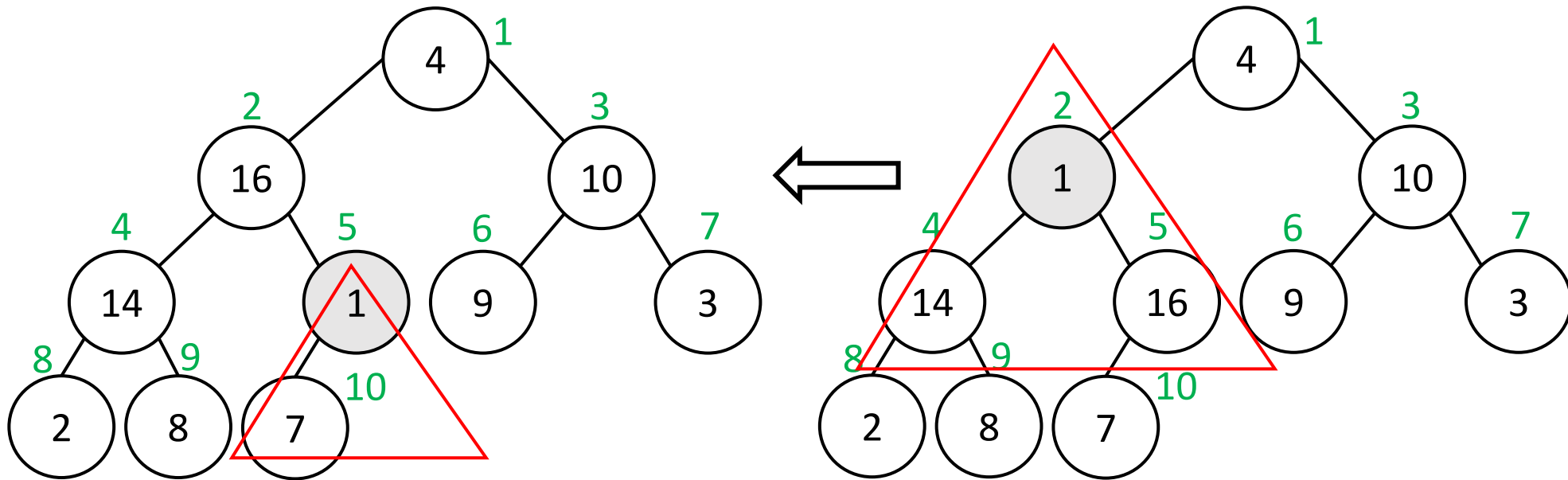
heapify(3)



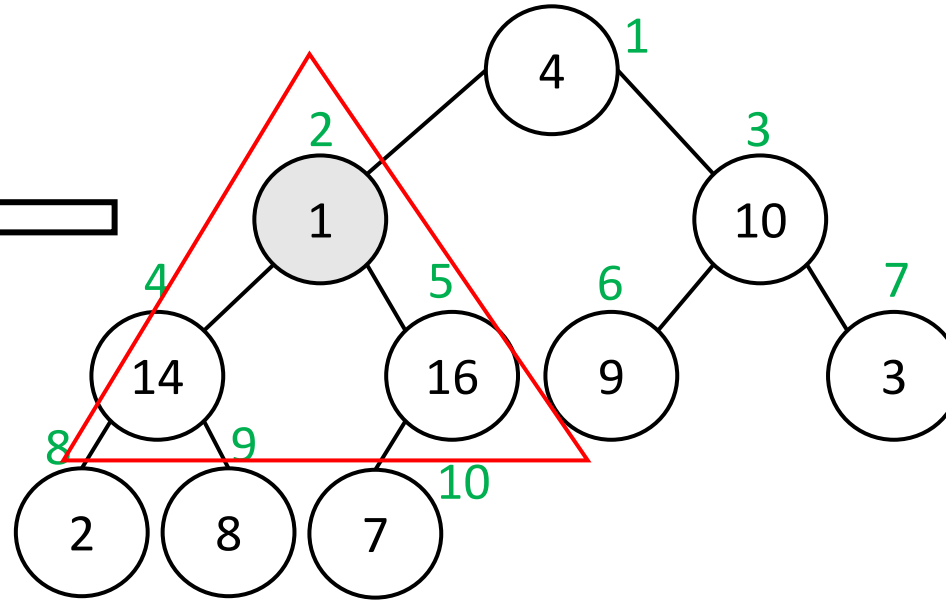
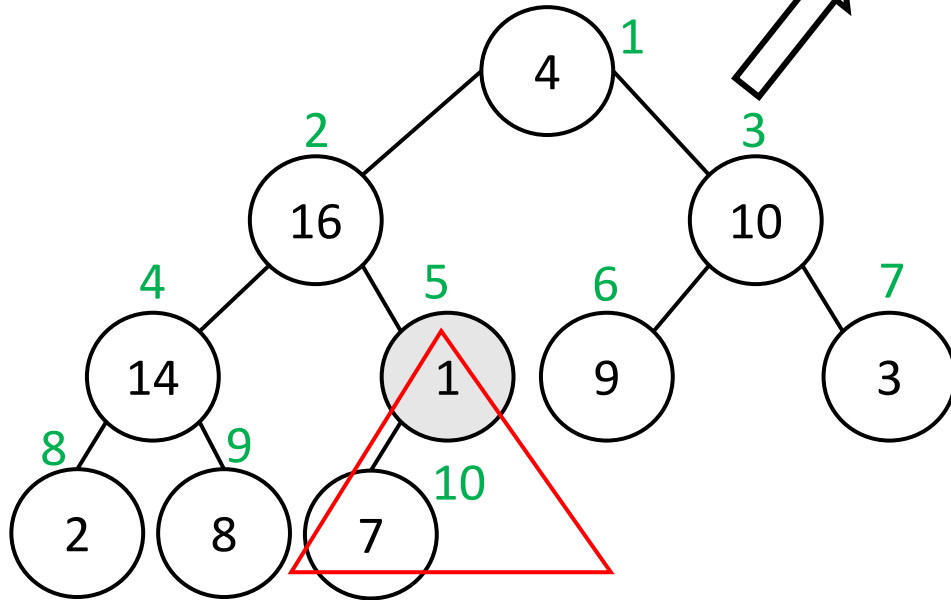
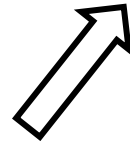
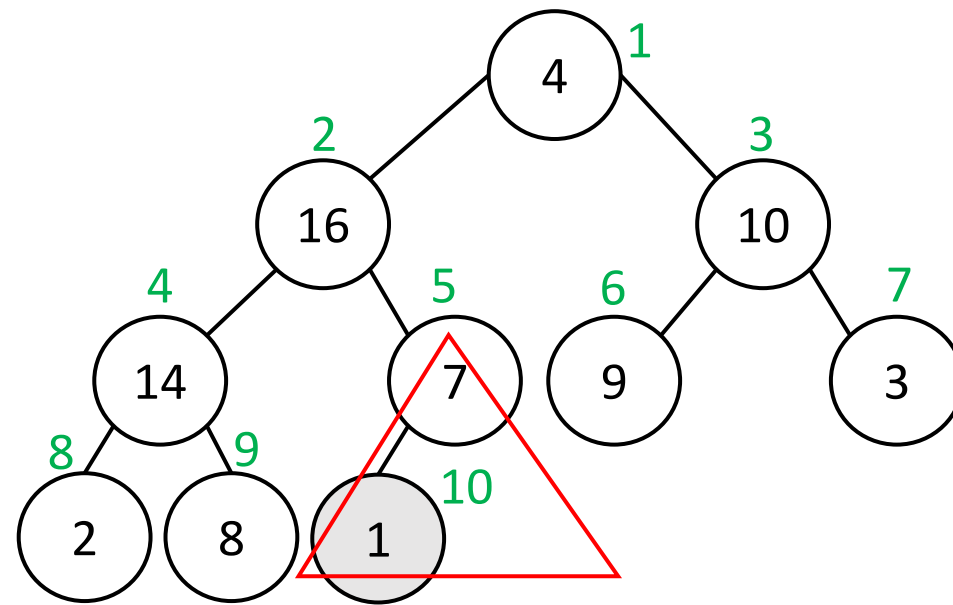
heapify(2)



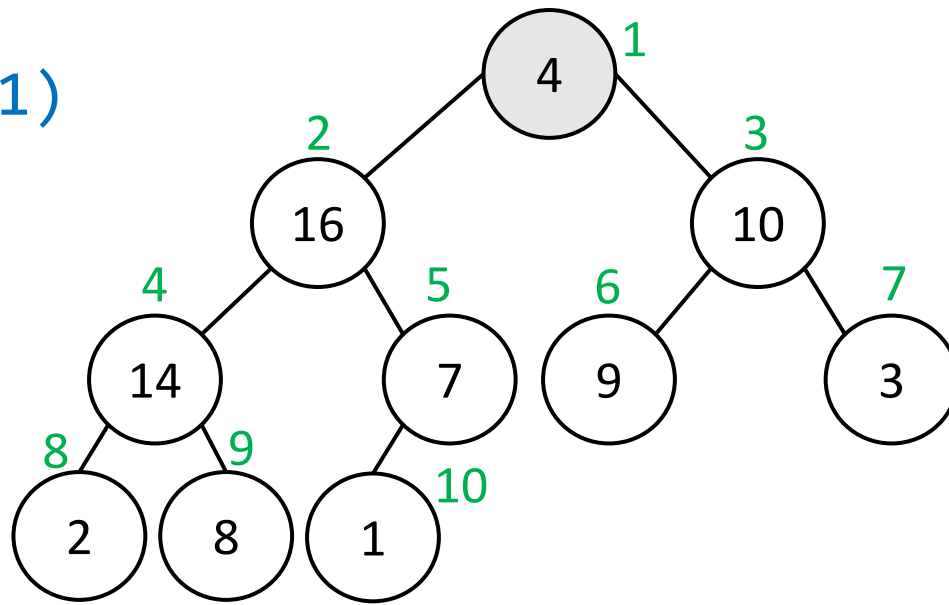
heapify(2)  
heapify(5)

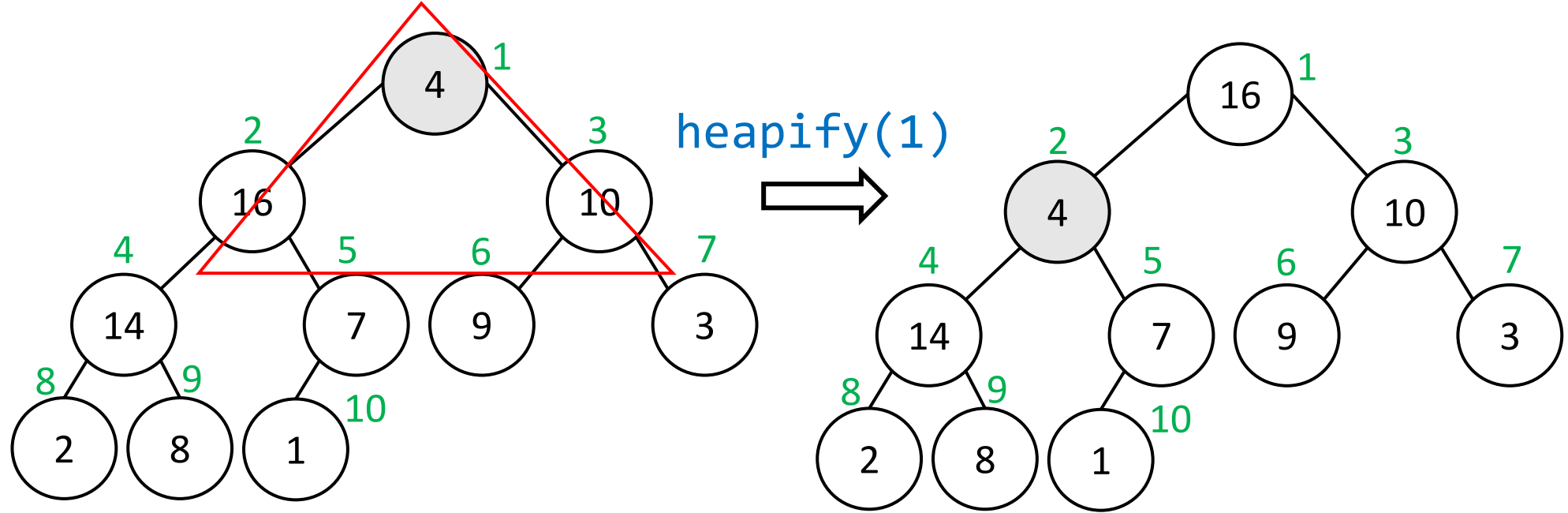


heapify(2)  
heapify(5)  
heapify(10)  
break

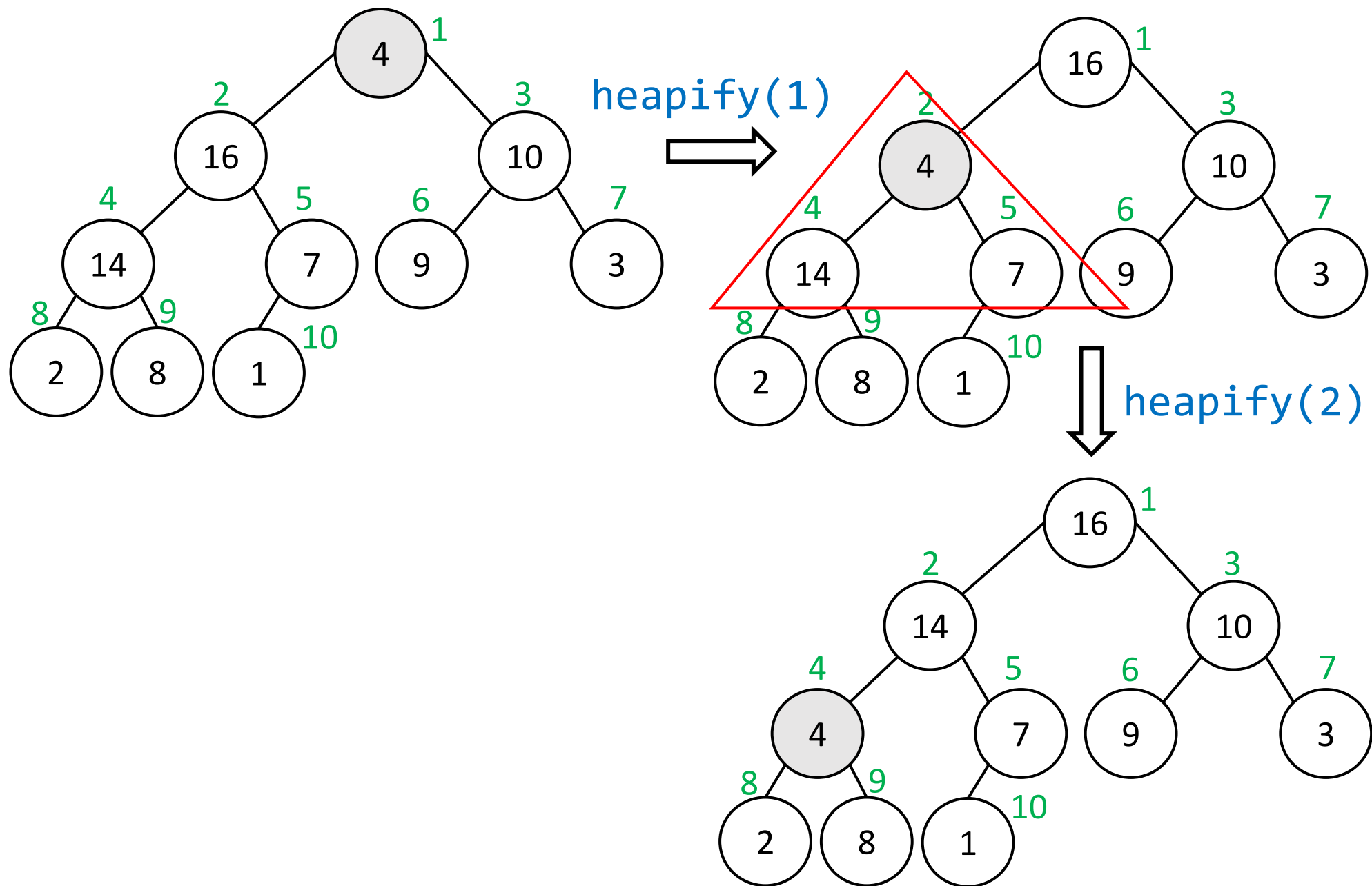


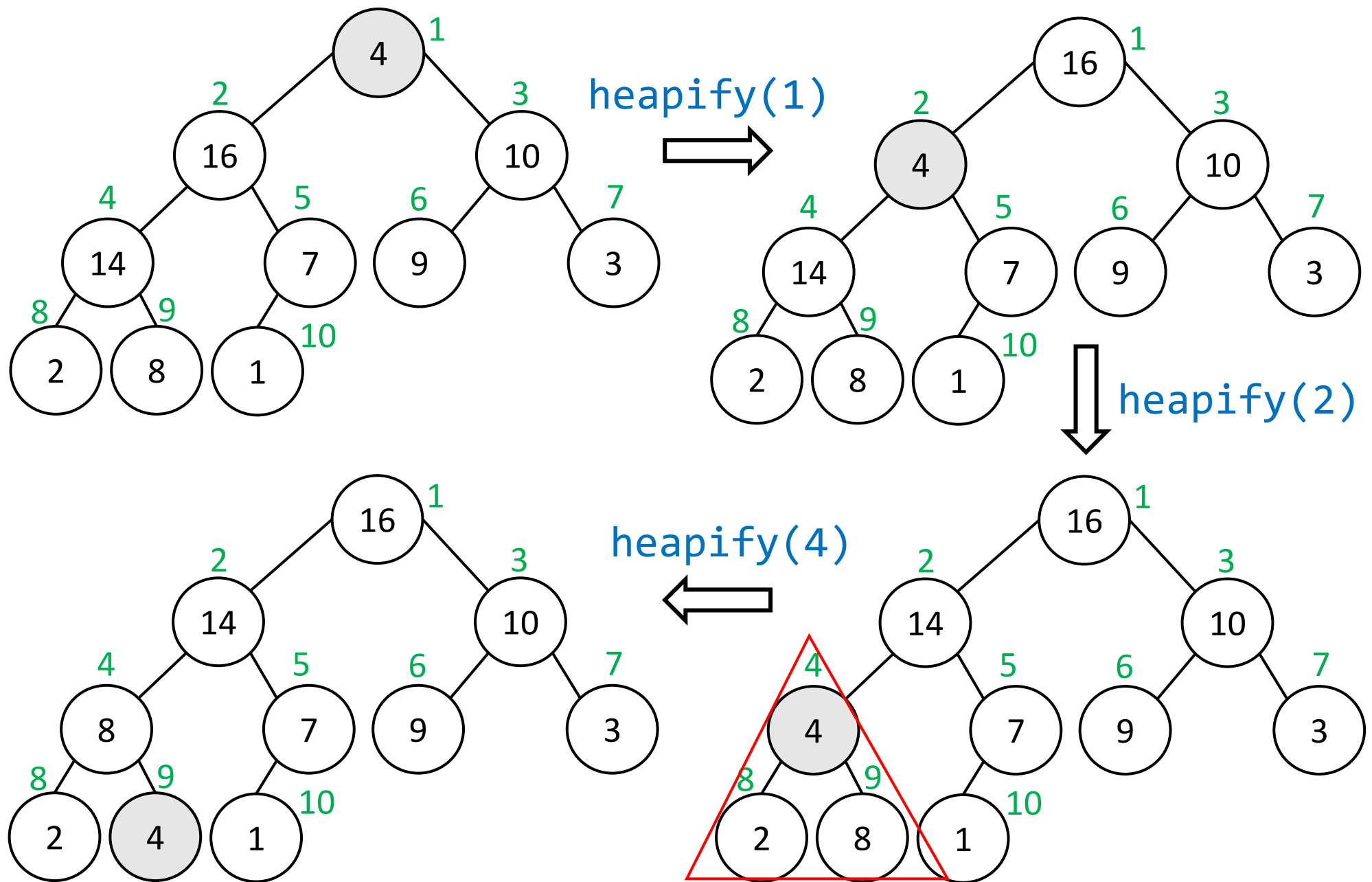
heapify(1)











```
build_max_heap({  
  
}
```

```
build_max_heap(){  
    for i=heapSize/2 downto 1  
  
}
```

```
build_max_heap(){  
    for i=heapSize/2 downto 1  
        max_heapify(i)  
}
```

```
build_max_heap(){  
    for i=heapSize/2 downto 1  
        max_heapify(i)  
}
```

*Time complexity?*

```
build_max_heap(){  
    for i=heapSize/2 downto 1  
        max_heapify(i)  
}
```

*Time complexity?*

$$O\left(\frac{N}{2} \log N\right) \sim O(N \log N)$$

```
build_max_heap(){  
    for i=heapSize/2 downto 1  
        max_heapify(i)  
}
```

*Time complexity?*

$$O\left(\frac{N}{2} \log N\right) \sim O(N \log N)$$

*It's a rough estimation.  
Detailed analysis shows that  
the complexity is  $O(N)$ .  
Next lecture!*



*Meme alert!*



# References

1. Chapter 6, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. Introduction to Algorithms, Third Edition (3rd. ed.). The MIT Press.
2. Wikipedia contributors. (2024b, June 20). *Binary tree* - Wikipedia.  
[https://en.wikipedia.org/wiki/Binary\\_tree](https://en.wikipedia.org/wiki/Binary_tree)

# Thank you